

Hierarchical Metric-Semantic SLAM Methods for Architectural Mapping

Jasper Wilkes



A dissertation submitted for the degree of
BSc Computer Science

Cardiff University

February 2026

Copyright © 2026 Jasper Wilkes

This work is licensed under the Creative Commons Attribution-ShareAlike License 4.0.

SPDX-License-Identifier: CC-BY-SA-4.0

Full text: <https://creativecommons.org/licenses/by-sa/4.0/legalcode>

Abstract

Simultaneous Localisation and Mapping (SLAM) is the fundamental robotics problem of estimating both the pose of a mobile robot and a map of its surroundings. Recent advancements addressing the **SLAM** problem use a graph-based representation of a scene that integrates metric (occupancy) and semantic (classes) estimates, along with hierarchical relationships connecting different aspects of the scene. This representation, known as a **3D Dynamic Scene Graph (DSG)**, encompasses actionable information that aids in robot planning and decision-making, but also in supporting architecture-related tasks such as renovation, inventory and remote building visualisation due its representation of building structures.

This project deploys and evaluates three state-of-the-art **DSG** methods: S-Graphs+ [3], Hydra [8] and Clio [12] in a simulated environment. The best performing method determined by the evaluation was processed on real-world data to create a **DSG** of the **Human Centred Computing and Robotics (HCCR)** laboratory. Each method uses a different approach to creating the **DSG**: S-Graphs+ is geometry based, Hydra adopts a Spatial approach while Clio uses a Task-Driven approach.

The evaluation revealed that S-Graphs+ struggled in cluttered environments, as objects like desks hindered wall plane identification, leading to incorrect classification and over-segmentation of rooms. Clio achieved high room recall (0.93) and correctly labelled all rooms, however over-segmented them with 0.45 Precision. A critical failure of Clio, despite it working on an example dataset, was it needing 0.25x playback speed to avoid missing frames or crashing. This failed the mandatory requirement of real-time processing, which eliminated it as a candidate for real world deployment.

Hydra stood out as the best performing method. It showed high room Precision 0.90, however was conservative with Recall 0.43. It exhibited strong object detection skills (F1: 0.62), and demonstrated consistent object localisation across runs (Avg 0.16m). Semantic labelling struggled with Weighted Average F1 of 0.35, with the method often labelling computers and monitors as "Thing", likely due to a label space mismatch.

Hydra was consequently used to create a **DSG** of the **HCCR** laboratory using real world data. A qualitative assessment showed that it correctly identified one room and correctly identified key objects within the lab like storage, wall decorations and workstation equipment.

Overall, Hydra has the potential to be useful for the architectural industry. The method's strong object detection could be used for inventory management tasks, while the high-precision room segmentation could be valuable for remote building visualisation. However, the method's conservative room segmentation and closed-set semantic limitations would need to be addressed for it to be more widely applicable.

Contents

Abstract	ii
List of Figures	vi
List of Tables	vii
Acronyms	viii
Dedication	ix
Acknowledgements	x
AI Tool Use and Acknowledgement	xi
1 Introduction	1
1.1 Motivation and context	1
1.2 Aims and objectives	2
1.3 Contributions	2
1.4 Dissertation structure	3
2 Background	4
2.1 Context	4
2.2 Related work	5
2.2.1 Building Visualisation	5
2.2.2 Evolution of Scene Representation	5
2.2.3 Ancestry Methods	6
2.3 Theory of Methods	6
2.3.1 S-Graphs+[3]	6
2.3.2 Hydra[8]	8
2.3.3 Clio[12]	10
2.4 Problem statement	11
2.5 Tabular Method Comparison	12
3 Methodology	14
3.1 Research design	14
3.2 Specification	14
3.2.1 Mandatory Requirements	14
3.2.2 Desirable Requirements	15
3.3 Candidate Methods	15

3.4	Comparative Evaluation Framework	15
3.5	Experimental Setup	16
3.5.1	Simulation	16
3.5.2	Real-World	18
3.6	Implementation	18
3.6.1	Simulation Environment Setup	18
3.6.2	Robot Configuration and Data Recording	19
3.6.3	Method parameters	20
3.6.4	Real-World Data Pipeline	22
3.6.5	Evaluation Scripts	22
4	Results and Evaluation	24
4.1	Simulation Output	24
4.1.1	S-Graphs+	24
4.1.2	Hydra	25
4.1.3	Clio	26
4.2	Simulation Results	27
4.2.1	Structural Consistency	27
4.2.2	Room Segmentation	28
4.2.3	Object Detection	29
4.2.4	Semantic Accuracy	30
4.2.5	Practicality	32
4.3	Simulation Discussion	32
4.3.1	Room Segmentation:	32
4.3.2	Semantic Labelling	34
4.3.3	Recording Technique	35
4.3.4	Practicality	35
4.4	Best-Performing Method Justification	36
4.5	Real-World Results	36
4.5.1	Graph Images	36
4.5.2	Mesh Reconstruction	38
4.5.3	Object Detection / Semantics	39
4.5.4	Room Segmentation	39
4.5.5	Practicality	39
4.5.6	Summary	40
4.6	Limitations	40
4.6.1	Dataset Context	40
4.6.2	Hardware	40
4.6.3	Recording Technique	41
4.6.4	Real-World Environment	41
4.6.5	Lack of S-Graphs+ Data	41
5	Conclusions and Future Work	42
5.1	Conclusions	42
5.2	Future work	43
6	Reflection on Learning	44

6.1	Critical narrative of the experience	44
6.1.1	Hardware Research	44
6.1.2	Environment Isolation	44
6.1.3	Community Insights	45
6.2	Analysis and application	45
6.2.1	Hardware Research	45
6.2.2	Environment Isolation	45
6.2.3	Community Insights	45
6.3	Implications for future practice	46
A	Supplementary Material	47
A.1	Additional tables	47
	License	51
	References	52

List of Figures

2.1	Pseudocode for novel Free-space Clustering Algorithm, reproduced from [3].	7
2.2	Illustration of room segmentation via graph of places dilation, reproduced from [8].	9
3.1	Reduced version of office scene from [4].	18
3.2	Robot used to record custom dataset.	18
3.3	Snapshot of main Hydra parameters which were tuned to better suit the custom office scene.	21
3.4	Region list for the custom office scene.	21
3.5	Task list for the custom office scene.	21
4.1	S-Graphs+ Output	24
4.2	Hydra Output	25
4.3	Clio Output with Mesh Layer Visible.	26
4.4	Clio Output with Mesh Layer Hidden (Different Run to Figure 4.3).	27
4.5	Hydra vs Ground Truth Semantic Labels: Normalised Confusion Matrix. Mean Confusion Matrix in Appendix Figure A.1.	30
4.6	Private Office Recognised as Room at First.	34
4.7	Private Office Room Node Merged into Open-Space Office.	34
4.8	Hydra 3D Scene Graph of HCCR Lab. Including Mesh Layer	37
4.9	Hydra 3D Scene Graph of HCCR Lab. Without Mesh Layer	37
4.10	Image of Storage Cabinets in HCCR Lab.	37
4.11	Snapshot of Hydra 3D Scene Graph of Storage Cabinets Section. (Taken from perspective of behind the wall in 4.10)	37
4.12	Image of Wall Decorations in HCCR Lab.	38
4.13	Snapshot of Hydra 3D Scene Graph of Wall Decorations Section. (Taken from perspective of behind the right wall in 4.12)	38
4.14	Image of Workstation in HCCR Lab.	38
4.15	Snapshot of Hydra 3D Scene Graph of Workstation Section. (Taken from perspective of behind the right wall in 4.14)	38
A.1	Mean Count Confusion Matrix of Hydra Labels vs Ground Truth Labels	48

List of Tables

2.1	High-Level Tabular Method Comparison	12
3.1	Comparative Evaluation Framework	16
3.2	Custom office rosbag ROS topics, their sources, and usage by methods.	20
4.1	Scene Graph Node Statistics.	27
4.2	Room Recognition Statistics. Best results in bold. Per-Run Statistics in Appendix Table A.2.	28
4.3	Object Detection Accuracy. Best results in bold.	29
4.4	Object Spatial Accuracy. Best results in bold. Per-Run Statistics in Appendix Tables A.3 and A.4.	29
4.5	Semantic Accuracy Statistics. Best results in bold.	31
4.6	System Timing Comparison across Stages (Seconds). Best results in bold. Per-Run Statistics in Appendix Table A.5.	32
4.7	Evaluation Hardware Specifications.	40
A.1	Semantic Label Mapping Rules	47
A.2	Raw Room Recognition Results over 5 Evaluation Runs	48
A.3	Raw Object Spatial Accuracy (Hydra vs Clio) across 5 Evaluation Runs	49
A.4	Detailed Object Drift Statistics (Hydra vs. Clio)	49
A.5	Raw Execution Timings across 5 Evaluation Runs (Seconds)	50

Acronyms

AIB Agglomerative Information Bottleneck. [i](#), [10](#), [11](#)

BIM Building Information Modelling. [i](#), [5](#)

CLIP Contrastive Language-Image Pre-training. [i](#), [11](#), [21](#)

DFS Depth-First Search. [i](#), [7](#)

DSG 3D Dynamic Scene Graph. [i](#), [ii](#), [1–5](#), [12](#)

GNN Graph Neural Network. [i](#), [9](#)

GVD Generalised Voronoi Diagram. [i](#), [9](#), [11](#)

HCCR Human Centred Computing and Robotics. [i](#), [ii](#), [2](#), [14](#), [19](#), [35](#), [41](#)

IAIB Incremental AIB. [i](#), [11](#)

IB Information Bottleneck. [i](#), [10](#)

IoU Intersection over Union. [i](#), [11](#), [16](#), [23](#), [29](#)

LiDAR Light Detection and Ranging. [i](#), [2](#), [5](#), [6](#), [12](#), [14](#), [15](#), [17](#), [18](#)

mED Mean Euclidean Distance. [i](#), [16](#), [29](#)

RANSAC Random Sample Consensus. [i](#), [6](#)

RGB Red Green Blue. [i](#), [18](#)

RGB-D Red Green Blue Depth. [i](#), [2](#), [4](#), [5](#), [8](#), [12](#), [14](#), [15](#), [17](#), [18](#)

ROS Robot Operating System. [i](#), [15](#), [18](#), [19](#)

SLAM Simultaneous Localisation and Mapping. [i](#), [ii](#), [1](#), [4](#), [6](#), [12](#), [14](#)

UK United Kingdom. [i](#)

URDF Unified Robot Description Format. [i](#), [19](#)

To my family.

Acknowledgements

I would like to thank my supervisor, Victor Romero Cano, for his guidance and support throughout this project.

I would also like to thank Emin Safa Tok for his assistance with learning how to use the ZED camera and providing the svo2 extraction script, which was crucial for processing the real world data.

Finally, I would like to thank my friends and family for their support and encouragement during this project.

AI Tool Use and Acknowledgement

To accelerate project infrastructure setup, four deployment infrastructure components were developed with assistance from Claude (Anthropic) [1], a ROS2 bag creation script (`svo2_to_rosbag.py`), a custom odometry-TF republishing node (`odom_to_tf.py`) and S-Graphs+ output parsers (`s-graph_to_json.py`) and (`extract_times.py`). This approach was justified to accelerate the setup of project-specific infrastructure which allowed more focus towards evaluation of the best-performing method. All AI-assisted code was manually reviewed and validated before use to ensure correct logic and implementation.

1. Introduction

This chapter introduces the project's motivation and context for the work, the aims and objectives, the main contributions, and the structure of the remaining chapters.

1.1 Motivation and context

SLAM is the fundamental problem in robotics of an online robot making a map of its surroundings (mapping), whilst simultaneously tracking its own position (localisation). This enables actions such as path planning. This presents a chicken and egg problem, you need the map to calculate position, yet need the position to build the map.

Classical approaches to the **SLAM** problem represent the environment using only metric terms such as occupancy grids, which captures the raw geometry without any structural meaning. Whilst this is enough for basic navigation, increasingly advanced robotics require a more structured understanding of the scene in order to execute higher-level functions. This is particularly apparent in architecture contexts, where tasks like inventorying or remote visualisation require an understanding of spatial structure.

As acknowledged by Topologic [9], current methods for building documentation rely on inferred geometry and do not explicitly represent spatial connectivity or enclosure. This limits real-world applications such as inventorying, which needs room-level semantic partitioning to associate objects with spaces. Existing solutions such as Topologic [10] acknowledges this problem, however it still depends on initial human input for the topology. This motivates an automated system for scanning a real environment to create a topological spatial model, which can then assist in the applications discussed above.

Recent advancements in **SLAM** research use a graph-based representation of the scene that integrates metric (occupancy) and semantic (classes) estimates, along with hierarchical relationships connecting different aspects of the scene. For example, an object belongs to a room. This approach, known as a **DSG**, enables it to have a broader range of higher-level tasks, such as responding to natural language commands like 'pickup the red pen'. These systems also have practical applications in the architectural industry. Identifying objects means it can be used for inventory tasks, and mesh reconstruction and hierarchical representation means it can be used for remote building visualisation.

Different **DSG** methods use different approaches to how the scene is processed. This means they differ in how the scene is constructed and labelled. This proposes three

core research questions: Which achieves the best accuracy, which is the most practical for deployment, and which creates the richest semantic understanding? This dissertation addresses these questions through a side-by-side evaluation of three state-of-the-art methods, resulting in deploying the best-performing method on real-world data.

1.2 Aims and objectives

The main aim of this project was to compare how three state-of-the-art methods perform according to different metrics, which would allow a best-performing to be selected to create a **DSG** of the **HCCR** laboratory. The project was broken down into four clear objectives:

O1: Deploy all three methods on example data

- Ensures that the methods are working as intended

O2: Deploy methods on custom rosbag¹ and store results

- Allows a side-by-side performance review between the methods

O3: Evaluate results and determine which is the best-performing method

- Determine the most suitable method for the **HCCR** lab in terms of metric, semantic and practical performance

O4: Use best-performing method on real-world data to generate a **DSG of the **HCCR** laboratory**

- Prove real-world practicality

1.3 Contributions

- **Integrated **RGB-D** camera onto existing 3D **LiDAR** robotic configuration**
 - Enabled multimodal data capture
- **Created simulated office dataset using teleoperated robot**
 - Created a rosbag recording of scene for reproducible evaluation
- **Deployed three state-of-the-art **DSG** methods (**S-Graphs+** [3], **Hydra** [8], **Clio** [12]) on the custom rosbag dataset**
 - Produced examinable results for evaluation
- **Evaluated methods based on accuracy, practicality, and richness of semantic understanding**
 - Enabled a systematic comparison and selection of the best-performing method

¹A file format in ROS for recording topic data and replaying it as real-time playback.

- **Deployed best-performing method on real-world data captured in the HCCR laboratory**
 - Generated a **DSG** of the scene and demonstrating the practical application of the method in a real-world context

1.4 Dissertation structure

The rest of this dissertation is organised as follows.

- **Chapter 2** (Background): Context, Related Work, Problem Statement.
- **Chapter 3** (Methodology): Research Design, Specification, Candidate Methods, Comparative Evaluation Framework, Experimental Setup, Implementation.
- **Chapter 4** (Results and Evaluation): Simulation Output, Results and Discussion, Best-Performing Method Justification, Real-World Results, Limitations.
- **Chapter 5** (Conclusions and Future Work): Conclusion, Future Work.
- **Chapter 6** (Reflection on Learning): Critical Narrative, Analysis and Application, Implications for future practice.
- **Appendix A**: Supplementary Material and Raw Results.

2. Background

This chapter provides context behind the project domain, a high-level comparison of the candidate methods, and the high-level theory and differences between each method. It ends with a clear problem statement which the project aims to address.

2.1 Context

Autonomous robots operating in dynamic environments require rich spatial representations to perform high-level tasks. Recent advancements in this topic have led to [DSGs](#) emerging as a way to build a hierarchical representation of a world scene, a key example being Hydra [8]. These scene graphs are split up into different layers, which each represent different aspects of the scene. For example, room and object layers contain information about specific detected rooms and objects like their position and semantic label. Layers are connected in a hierarchical structure which reflects the structure of the environment. For example, building nodes contain floor nodes, which contain room nodes, which contain object nodes.

This structured approach allows a more contextual view of an environment. Giving a robot more context of its surroundings enables them to perform higher-level tasks. One application for this concept is architectural-related tasks. For example, if a robot can detect objects throughout a building, it can then be used to conduct an inventory of specific items like chairs or pens, which can further be adapted to aid in building management. Another benefit could be assisting architects. A [DSG](#) produces an abstractable view of a building, which can be used by architects to analyse relational data of the scene to aid them in further development.

The general pipeline for a [DSG](#) method starts with a metric layer, where the respective sensor data is turned into a volumetric or mesh representation, this forms the metric layer of the graph and is the minimum expected for a [SLAM](#) solution. Then, the method will analyse the metric layer to extract information about the scene. The way this is extracted depends on the method and graph structure being used. For example, S-Graphs+ [3] analyses a point cloud to segment walls using the geometry of the scene, whereas Hydra [8] uses neural networks to extract objects from [RGB-D](#) data. Each layer is constructed using the data of the layers below to eventually build a graph encompassing the whole scene.

2.2 Related work

2.2.1 Building Visualisation

Topologic [9] is a software toolkit which integrates several architecturally relevant topological concepts to create hierarchical and topological information-rich 3D representations to aid in the design process. Its goal is to support the creation of the lightest, most understandable models of architectural topology. The method addresses a limitation of conventional [Building Information Modelling \(BIM\)](#) applications (not explicitly modelling the enclosure of space) by using formal topology to represent spatial and material partitions.

However, this system is limited by its requirement for clean geometry input in order to process the data. It relies on a CellComplex, a structure where every spatial cell is perfectly enclosed and shares boundaries with its neighbours. It is rare for real-world sensors like 3D [LiDAR](#) and [RGB-D](#) cameras to capture data this complete, as they usually result in unstructured noisy data, which naturally creates the need for extra steps between data collection and topological analysis.

[DSGs](#) bridge this gap by automating the transition from data collection and topological representation. [DSGs](#) use room segmentation and semantic partitioning techniques to automate transforming raw data into a hierarchical view with semantic understanding which is useful for architectural tasks like inventorying or remote building visualisation.

2.2.2 Evolution of Scene Representation

Early work in scene representation focused on static environments containing geometric and occupancy information about the scene. An example of this is OctoMap [5], which is a volumetric 3D environment model using octrees and probabilistic occupancy estimation, which allows occupied and free or unknown space. Whilst this is useful for low-level tasks like basic navigation and mapping, it lacks the depth and understanding required for high-level tasks such as inventorying.

Recent developments have shifted to hierarchical models which capture semantic relationships within the scene. S-Graphs+ [3] introduced room-level topology, capturing the geometric layout of a scene while identifying for example which rooms are next to which and what floor are they on. This concept has since expanded to systems such as Hydra [8] which incorporated object detection with closed-set semantics, to then Clio [12] which introduced open-set semantics. These systems represent the state-of-the-art in turning raw sensor data into graphs with architectural understanding, which allows for higher-level tasks such as inventorying or remote building visualisation.

2.2.3 Ancestry Methods

Pioneering scene graph work included the 3D Scene Graph [2], this system takes a 3D mesh and registered panoramic images and constructs a graph that includes semantics, rooms and cameras (location) along with relationships between these entities. However, this is a static system which requires preprocessed input to be ran offline, which eliminates real-time use for tasks such as remote building visualisation. This is a contrast to the candidate methods, as a mandatory requirement specified in Section 3.2 is to be able to run in real-time.

The introduction of the open-source library Kimera [19] enabled simultaneous metric semantic mapping. This library extends previous SLAM libraries by enabling 3D mesh reconstruction and semantic labelling. This library became the foundation for systems such as the candidate method Hydra [8] which builds on this by implementing higher-level hierarchical perception.

2.3 Theory of Methods

2.3.1 S-Graphs+[3]

S-Graphs+ jointly models a pose graph and a 3D scene graph using a four-layer topological graph consisting of Keyframe, Wall, Room and Floor layers. Unlike the other two methods, its data input is from a 3D LiDAR sensor, which is pre-filtered and down-sampled before processing. Robot odometry (used for SLAM) is estimated either from LiDAR measurements or robot encoders, the method is agnostic to which it comes from.

Wall Extraction

Sequential Random Sample Consensus (RANSAC) is applied per keyframe to iteratively detect dominant wall planes. RANSAC works like a voting system to decide which points belong to the same flat surface. Only processing keyframes instead of every point of data collected is vital for computational performance. Analysing consecutive scans would likely result in the same detections, therefore processing per keyframe ensures meaningful changes are captured without processing unnecessary data. Extracted walls are transformed into global map coordinates and classified into vertical (walls) and horizontal (floor/ceiling) surfaces using their normal vector orientation. As the robot traverses the scene, the Mahalanobis distance, which is a statistical measure that accounts for uncertainty, is used to detect whether a plane is new or has been previously discovered.

Room Segmentation

This method introduces a novel strategy for segmenting rooms. It is split into two steps: Free-Space Clustering and Room Extraction.

Free-Space Clustering:

The free-space clustering algorithm (Figure 2.1), divides the free-space graphs into several clusters corresponding to rooms. The algorithm has three main steps:

1. **Pruning:** Removes boundary nodes which are too close to walls or obstacles. This helps to separate rooms by creating gaps such as doorways to segregate room clusters.
2. **Clustering:** A [Depth-First Search \(DFS\)](#) is used to find isolated islands of connected nodes, which become clusters. Since doorways were pruned in the previous step, these clusters are assumed to represent rooms.
3. **Restoration:** Previously removed boundary nodes are re-added to maintain full coverage of free-space. This is vital for actions such as navigation.

Algorithm 1: Free-Space Clustering

Input: Free-space graph $\mathcal{G}$, generated using [24]
Output: Clustered sub-graphs $\mathcal{G}_{f_i}$, $i \in \{1, \dots, N\}$

1: Filter nodes far from obstacles: $\mathcal{G}_f \leftarrow \mathcal{G}$

```

for  $v_i \in \mathcal{V}$  &  $v \in \mathcal{V}$  do
  if  $v_i.distance < t_\lambda$  then
     $v_d \leftarrow v_i$  &  $e_d \leftarrow v_i.edges$ 
  else
     $v_f \leftarrow v_i$  &  $e_f \leftarrow v_i.edges$ 
  end
end
 $\mathcal{G}_f \leftarrow (\mathcal{V}_f, \mathcal{E}_f)$ 

```

2: Graph clustering by connectivity: $\mathcal{G}_{f_1} \dots \mathcal{G}_{f_n} \leftarrow \mathcal{G}_f$

```

 $c \leftarrow 0$ 
for  $v_{f_i} \in \mathcal{V}_f$  do
   $v_{f_i}.visited \leftarrow False$  &  $v_{f_i}.cluster \leftarrow 0$ 
end
Function visit( $v_f$ ):
  if  $v_{f_i}.visited = False$  then
     $v_{f_i}.visited \leftarrow True$ 
    if  $v_{f_i}.cluster = 0$  then
       $c \leftarrow c + 1$ 
       $v_{f_i}.cluster \leftarrow c$  &  $\mathcal{G}_c \leftarrow v_{f_i}$ 
    end
    for  $v_{f_n} \in v_{f_i}.neighbours$  do
       $v_{f_n}.cluster \leftarrow c$  & visit( $v_{f_n}$ )
    end
  end
end
return
for  $v_{f_i} \in \mathcal{V}_f$  do
  visit( $v_{f_i}$ )
end

```

3: Inclusion of deleted vertices and edges in the cluster: $\mathcal{G}_{f_i} \leftarrow (\mathcal{V}_d, \mathcal{E}_d)$

```

for  $v_{d_i} \in \mathcal{V}_d$  do
  for  $v_n \in v_{d_i}.neighbours$  do
    if  $v_n \in \mathcal{G}_{f_i}$  then
       $c \leftarrow v_n.cluster$  &  $\mathcal{G}_{f_i} \leftarrow (\mathcal{V}_{d_i}, \mathcal{E}_{d_i})$ 
    end
  end
end

```

Figure 2.1: Pseudocode for novel Free-space Clustering Algorithm, reproduced from [3].

Room Extraction:

This step combines identified clusters with detected walls within that cluster to classify room configurations. Wall planes are categorised by their dominant normal direction and the sign of the normal. This allows the system to see which walls face each other and whether they form rooms.

Rooms are classified as either two or four wall rooms. A four-wall room consists of two x and two y direction planes near a cluster, which is then verified by checking its width to prevent noise being classified as rooms. If two walls surround a cluster, it is either a partially explored room or a corridor-like structure, for which the same verifications are ran. Data association steps include using room centres to identify whether multiple candidates represent the same room. The Mahalanobis distance is also used in cases where a wall might be observed different from different viewpoints.

Floor Segmentation

This module extracts the widest wall planes within the current explored floor which can be used to determine a floor centre. It does this by using all stored walls to compute widths between all x and y direction planes. The largest of each orientation undergo a dot product check between their corresponding normal orientations which removes planes which are outside the structure. When the robot ascends or descends to a different floor, the new wall planes are stored corresponding to the floor level they are on.

Back-End

The graph is created and optimised in the back-end by minimising errors across all layers, refining both the robot's pose estimates and the structure representations simultaneously.

2.3.2 Hydra[8]

Hydra combines fast early and mid-level perception processes like local mapping with slow higher-level perception like global optimisation of the scene graph. Unlike S-Graphs+, Hydra grounds symbolic knowledge into the geometric map to create metric-semantic representation. This is useful for complex instructions which need contextual information about the environment.

The hierarchical graph contains five layers: Metric-Semantic 3D Mesh, Objects and Agents, Places, Rooms, Buildings. When compared to S-Graphs+, Hydra [8] represents a much more grounded and actionable representation. The nodes must contain a single parent in the layer above, they can also only connect to adjacent layers, e.g. a chair connects to a place node, but not a building. Sibling nodes children are disjoint, meaning furniture in sibling rooms are completely independent of each other. Hydra uses tree decomposition to represent a complex graph as a tree structure by using bags, which contains a set of nodes from the original graph. This stops the complexity exploding with size, bounding it to the most complex local structure for example how many objects in a room.

Mesh

A 3D mesh is built using Kimera [19] and VoxBlox [15] by maintaining a voxel-based map within an active window around the robot. Each voxel contains information about the free space and a distribution over semantic labels via Kimera's Bayesian inference. This map is then converted to a 3D mesh using marching cubes, with a semantic label added to each mesh vertex. Operating within a specified active window is useful for memory usage as it limits the amount of data trying to be processed at once.

Objects and Agents

This layer consists of a robot pose graph obtained using stereo or RGB-D visual inertial odometry. It also segments objects using Euclidean clustering, where vertices with the same semantic label are clustered. Overlapping objects with the same semantic class are merged if the centroid of an object is within another's bounding box.

Places

A [Generalised Voronoi Diagram \(GVD\)](#), represents a skeleton of free-space. It is created using a Brushfire algorithm by expanding from each mesh surface until touching, which becomes a [GVD](#) point. Whilst this captures the full shape of the free-space, it is heavy to store due to its voxel representation. So, each voxel is translated to a node with edges to adjacent voxels, groups of nearby nodes are then turned into single “place” nodes which is more memory efficient. Each node contains the position and distance to nearest obstacle from the [GVD](#) voxel, as well as the minimum distance to the nearest obstacle of all [GVD](#) voxels that the edge passes through. This information is useful for navigation tasks. As the robot traverses the scene, the algorithm is run after every update to merge newly observed data into the graph of places.

Rooms

To identify rooms, persistent homology [7] is used on the graph of places, then each room is then given a semantic label using a neural tree. For the semantic labelling, we don’t have a network to identify rooms like we do for objects. Therefore, the semantics of the room must be inferred using contained objects and prior knowledge. Rooms are identified using a 2 stage process, identifying number of rooms, and assigning places to rooms via flood-fill. The graph of places is dilated (Figure 2.2) to expose clusters of nodes in the environment (which represent rooms). This works as eventually door nodes will gradually close, naturally sectioning the map into rooms.

Persistent Homology is used to compute the best dilation distance for a given graph, this allows room segmentation to be effective across different scene contexts. The Flood-Fill algorithm is then used to assign remaining nodes to the putative room. Semantic labels are assigned using a neural tree. The key insight of this process is objects which are typically associated with a room type, e.g. a fridge. Edges may also be used such as a master bedroom being next to a bathroom.

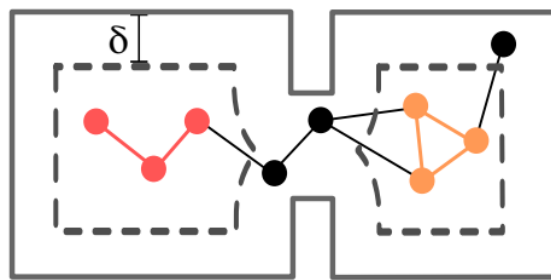


Figure 2.2: Illustration of room segmentation via graph of places dilation, reproduced from [8].

Loop Closure Detection

For each keyframe, it tries to find a past node which matches the latest one using associated Places, Objects and Appearance descriptors. Descriptors can be Hand-Crafted or Learning-based, which uses [Graph Neural Network \(GNN\)](#)s. The hand-crafted method counts object labels and uses a histogram of distance to obstacles for

each place node, whereas the learning-based method uses bounding boxes, labels and spatial relationships for objects and distance and encodes distances between nodes to make it rotation invariant. Once a match candidate is found, it is verified using a top-down then bottom-up strategy, this makes sure false loop closures are avoided.

Optimisation

Odometry drift errors increase as the robot explores more of the scene, therefore the scene graph must be updated often to stay consistent. Hydra uses a sparse sub-graph called a deformation graph which has robot poses, mesh control points and a sparse version of all place nodes, this representation is more efficient than optimising the entire graph every time. Each node in this graph has a local coordinate frame (position + rotation), the optimiser then finds poses which minimise deformation by respecting original measurements but allowing for some flex. You can think of it like the graph edges being rubber bands, the optimiser pulls them to satisfy loop closures while minimising how much it deforms. Once the optimiser is finished, the corrected deformation graph is interpolated to reconstruct the full scene mesh and then reconciles the graph by merging place nodes within 0.4m of each other, and objects with same semantic labels and invading bounding boxes.

2.3.3 Clio[12]

Clio is the most unique out of the three methods discussed in this project. The other methods build task-agnostic graphs, meaning they capture everything about the scene regardless of the robots intentions. Clio differs from this by creating a task-driven graph which only captures aspects of the scene which is relevant to the tasks the robot needs to carry out. This can be a more efficient representation as it only stores/processes things that are directly relevant, however this means the built graph cannot be used for more general operations.

Task-Aware 3D Scene Understanding

A fundamental concept for Clio is task-aware 3D scene understanding. It involves compressing a set of task-agnostic primitives X into a subset of task-relevant concepts $\tilde{X}$ that are informative about the tasks Y . A naive approach would cluster segments based on embedding vectors exceeding a threshold. This would require multiple tuning thresholds such as number of segments or merge similarity, which are brittle and scene-specific.

This naturally leads to the [Information Bottleneck \(IB\)](#) principle, which aims to compress a given signal while preserving the mutual information between the compressed representation and another signal of interest. Clio uses this principle with the context of compressing a scene representation into clusters of relevant objects and regions for a given set of tasks, replacing thresholds with a task-driven objective.

Agglomerative Information Bottleneck (AIB)

[AIB](#) is a bottom-up merging approach, meaning it starts with all individual primitives, then iteratively merges adjacent ones. Cluster merges are decided using a task-driven

metric by computing the weight for each possible merge using the Jensen-Shannon divergence [6], then iteratively merge clusters corresponding to the smallest weight. During each iteration, a value $\delta(k)$ is calculated as a measure for the loss of information. The algorithm is terminated once the threshold for this value is exceeded which stops losing too much information through abstraction.

Clio develops AIB further with [Incremental AIB \(IAIB\)](#) for real-time application. As the graph grows over time, computational complexity must be bound to maintain efficiency. [IAIB](#) handles this by clustering independently for each connected component. For example, objects in different rooms have no edges, so they can't merge anyway. This method allows $\delta(k)$ to be calculated independently for each component which maintains accuracy while bounding computational complexity.

[AIB](#) requires defining a relevance probability for each primitive, for which Clio uses [Contrastive Language-Image Pre-training \(CLIP\)](#). [CLIP](#) embeds both primitives and task descriptions into a shared semantic space. Then a cosine similarity score can give task-relevance per primitive. A null task with threshold α handles irrelevant primitives via pre-pruning before [AIB](#) runs. This is efficient as it reduces the graph size before the merging step which is expensive.

Front-End

Clio's front-end constructs task-agnostic object and place primitives using Khronos [20] for 3D mesh reconstruction and primitive extraction. FastSAM and [CLIP](#) get semantic segments for each image. Each new segment is greedily associated to the candidate with the highest [Intersection over Union \(IoU\)](#), with no association creating a new track. Then, each track is reconstructed into a 3D primitive based on all the frames in the track, which gives a more robust semantic embedding than using one frame alone. Simultaneously, the background is constructed for each frame, which allows for a dense 3D model to be incrementally constructed with limited computation. To construct place primitives, it follows Hydra's approach of sparsifying a [GVD](#) into a graph of places. Then a [CLIP](#) embedding vector is used to obtain semantic features for the place.

Back-End

The back-end takes the over-segmented primitives from the front-end and applies [AIB](#) to cluster them based on their task relevance. During object detection, primitives are treated as nodes, with edges defined by bounding box overlaps which ensures only physically adjacent segments are candidates for merging. When clustering places, it applies the same principles to the places graph, grouping nodes into semantic regions based on their relevance to tasks. For example a sink and stove clustered into a kitchen region.

2.4 Problem statement

Ideally, all [DSGs](#) would be able to capture highly accurate representations of the environment that also captures architectural topology and semantics of the scene.

However, current Metric-Semantic SLAM methods all excel and struggle at different aspects, such as metric accuracy vs semantic accuracy.

Whilst state-of-the-art methods have rigorous testing on datasets tailored for their own method, there is limited side-by-side evaluation in a fair and controlled environment. Without this, the architectural industry cannot determine which is the most reliable and suitable to aid in architectural applications such as inventorying or remote visualisation.

This dissertation bridges the gap by deploying each method on the same dataset. This will help identify the strengths and weaknesses side-by-side in order to reflect on their usage potential for architectural tasks.

2.5 Tabular Method Comparison

Table 2.1 gives a side-by-side comparison of the candidate methods. It outlines the graphs structure, data processing approach, data input type and semantic approach.

Table 2.1: High-Level Tabular Method Comparison

Method	Layers No.	Layer Names	Data Approach	Data Input	Semantics
S-Graphs+	4	Keyframes, Walls, Rooms, Floors	Geometric	LiDAR	-
Hydra	5	Metric-Semantic 3D Mesh, Objects and Agents, Places, Rooms, Buildings	Spatial	RGB-D	Closed-Set
Clio	5	3D Mesh, Objects and Agents, Places, Rooms, Buildings	Task-Driven	RGB-D	Open-Set

Layer Descriptions:

- **Keyframes:** Robot pose estimates
- **Metric-Semantic 3D Mesh:** Dense 3D reconstruction of the scene with semantic labels
- **Walls:** Planar structural surfaces
- **Objects:** Semantic objects within the scene, and robot pose estimates
- **Agents:** Robotic Agent pose estimates
- **Rooms:** Enclosed segmented regions
- **Places:** Graph representing free-space (for navigation)
- **Floors:** Floor levels containing Rooms

- **Buildings:** Encapsulates all rooms and floors within a building

3. Methodology

This chapter describes the evaluation design, specification, justification for the inclusion of each candidate method, metric framework evaluated, the experimental setup for both simulation and real-world deployment, and the implementation conducted.

3.1 Research design

This project presents a comparative evaluation across three state-of-the-art candidate methods: S-Graphs+[3], Hydra[8] and Clío[12].

The methods were evaluated over five executions on a controlled and consistent simulation environment. The highest-performing method was determined through evaluation of three main categories: Metric accuracy, Semantic accuracy and Practical feasibility. This chosen system was then deployed on real-world sensor data of the HCCR laboratory.

Each method was deployed on the same simulated environment. The sensor data was recorded in a single rosbag using a RGB-D camera (Hydra and Clío) and 3D LiDAR sensor (S-Graphs+), which were attached to a differential drive robot and teleoperated around the scene. The methods resulted in a 3D scene graph, which was then saved in JSON format for further statistical evaluation.

Simulation results were evaluated quantitatively to determine the highest-performing statistically, then the real-world results were evaluated qualitatively. This is because ground-truth¹ is accessible for Gazebo [16] simulation environments, however generating ground-truth can be complicated for real-world scenes.

3.2 Specification

To evaluate the suitability of the candidate methods for architectural tasks, requirements are categorised into Mandatory and Desirable requirements. These define the fundamental capabilities for SLAM operations and the ideal capabilities for higher-level architectural tasks such as inventorying.

3.2.1 Mandatory Requirements

- **Hierarchical Scene Representation:** The method must generate a 3D scene graph with multiple abstraction layers to provide architectural context.

¹True, known values used to measure the accuracy of the system's estimates

- **Spatial Segmentation:** The method must be able to segment environments into distinct room nodes.
- **ROS Compatibility:** The method must be compatible with [Robot Operating System \(ROS\)](#) input data.
- **Operational Stability:** The method must be able to process the office scene without fatal errors.
- **Quantitative Evaluation:** The evaluation framework must calculate Recall, Precision, and F1-score for room nodes, and object nodes where applicable.

3.2.2 Desirable Requirements

- **Semantic Object Layer:** The system should provide a semantic object layer to support automated inventorying or interactive tasks.

3.3 Candidate Methods

The three candidate methods are S-Graphs+[3], Hydra[8] and Clio[12]. Below is a brief explanation of their main characteristics and why they were chosen for the evaluation. More detailed explanations of each method can be found in Section 2.3.

S-Graphs+ takes 3D [LiDAR](#) point cloud as input. It then analyses the point cloud to identify geometric planes, which are then used to detect rooms. There is no object segmentation or semantic labelling, its output mostly contains navigation and structural information. This method was selected to observe how its purely geometric structure would compare to methods which also incorporate semantics.

Hydra adopts a spatial approach using [RGB-D](#) data to reconstruct a 3D metric-semantic mesh of the scene. It segments rooms using free-space graph dilation. Hydra differs from S-Graphs+ by including an object layer, as well as using a neural network to assign semantic labels throughout the graph. This method was included to evaluate the spatial approach compared to geometric and task-driven approaches.

Clio adopts a task-driven methodology rather than being task-agnostic like the other candidates. Clio's semantics are open-set compared to Hydra's closed-set approach. Room segmentation is conducted by grouping semantic objects into regions rather than purely using geometric boundaries. This method was chosen to evaluate how task-driven approaches compare to task-agnostic methods.

3.4 Comparative Evaluation Framework

As stated above, the evaluation criteria aimed to address three sections: Metric accuracy, Semantic accuracy and Practical feasibility. These sections were split up into specific and comparable metrics stated in Table 3.1 below:

Table 3.1: Comparative Evaluation Framework

Category	Metric	Purpose
Metric: Structural	Node Count (Total, Room, Object)	Assesses structural reproducibility across runs
	Room Count	Assesses room detection accuracy
	Room Recall, Precision, F1	Measures room completeness & accuracy
Metric: Objects	Recall, Precision, F1	Measures completeness & accuracy of object detection
	Per Run Object Drift	Measures object localisation consistency
	Centroid Mean Euclidean Distance (mED)	Measures spatial accuracy of object localisation
	IoU Bounding Box Overlap	Measures spatial accuracy of detected bounding boxes
Semantic	Object Confusion Matrix	Assesses per-class labelling accuracy and consistency
	Label Distribution	Assesses semantic completeness for object detection
Performance	Front-end / Back-end Timing	Measures run-time performance
	Real-time Capability	Assesses viability for real-world deployment

3.5 Experimental Setup

This section explains the approach to the simulation and real-world evaluation. It includes how the datasets were recorded, the evaluation stages, and the processing requirements.

3.5.1 Simulation

The custom dataset used for the evaluation was the office scene shown in Figure 3.1, recorded in Gazebo using a differential drive robot shown in Figure 3.2. The robot was configured to mimic the sensors on a Reachy2 [18] robot, which represents a realistic candidate for indoor tasks due to an omnidirectional base and gripper hands. It had a velodyne 3D [LiDAR](#) sensor (for S-Graphs+) attached to the base with a [RGB-D](#) cam-

era (Hydra and Clio) attached to the body halfway up. The camera was a simulated camera setup to emulate the Intel Realsense D435 camera due to it being popular in robotics research, recording 1280x720 image at 30Hz, with 1.5708 Field of View. The office scene was sourced from [4], which was segmented to include only a portion of the environment shown in Figure 3.1. The decision to use a portioned section of the scene was made to ensure the recording was a feasible length whilst including multiple room types and objects.

Each method was deployed on the same rosbag recording (converted to ROS1 bag for Clio), this made the evaluation fair, as it ensures they all process the same data, which allows for a direct comparison.

Before testing commenced on the custom dataset, the methods were verified against example/known data to ensure they had correct behaviour. Hydra and Clio were tested on their respective example datasets. However, there was no provided example data for S-Graphs+, but it was deployed on a small recording of an indoor house from a prior 'Introduction to Computational Robotics' module, which provided a simple environment to assess expected behaviour. Once expected behaviour was confirmed, the evaluation stage could then begin.

A key deployment limitation was observed with Clio. The rosbag image data had to be downsampled and played at maximum 0.25x speed in order for frames to be processed correctly and the system not to crash. Even with this input, GPU usage was observed to spike to 98% occasionally, which suggests the bottleneck might have been to do with the hardware gap shown in Table 4.7. Notably, Clio ran without issue on the example dataset at real-time playback, suggesting the limitation is a specific response to the custom dataset. This represents a real-world capability limitation, as Hydra was able to run in real-time playback with the original image size without issue.

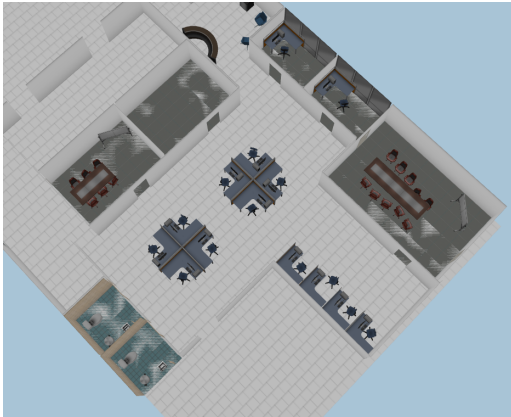


Figure 3.1: Reduced version of office scene from [4].

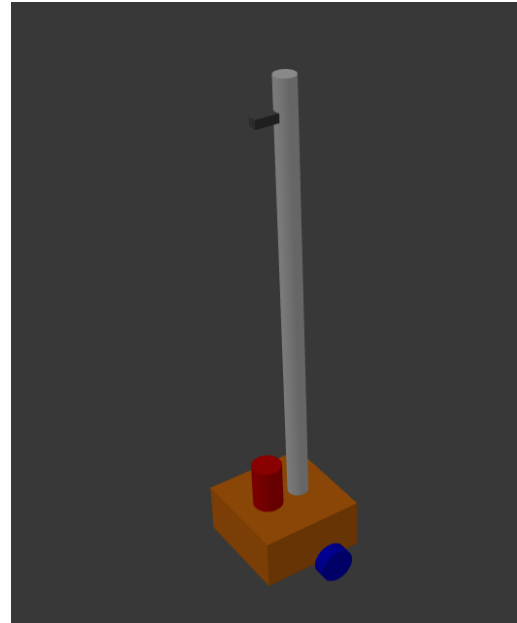


Figure 3.2: Robot used to record custom dataset.

3.5.2 Real-World

The simulation phase concluded that Hydra was the best-performing method as an all round system (discussed in Section 4.4). Therefore, this meant 3D LiDAR data wasn't required for the real-world deployment due to S-Graphs+ being excluded from this stage.

The ZED camera was the optimal choice for recording the scene due to its simple setup and RGB-D capabilities alongside pose estimates, it was set to record 720p quality at 30Hz, which enabled sufficient quality and coverage. To mimic the perspective of a traversal robot like Reachy2, The camera was handheld at torso height (approx. 1.2m) whilst being walked slowly around the HCCR lab.

As the ZED camera naturally lacks ROS message outputs, the svo2 formatted video was extracted into RGB and Depth frames along with pose history of the robot. This data was then encoded into a ROS2 bag recording which can be processed by Hydra.

3.6 Implementation

3.6.1 Simulation Environment Setup

Each method requires different ROS environments. There was no ROS version which was supported by all three methods simultaneously. To ensure a smooth deployment of all three methods, Docker was used to help maintain three separate environments.

S-Graphs+ was installed natively with ROS2 Humble as it was the most lightweight of the candidate methods and didn't require any major dependencies like CUDA. To

ensure a smooth installation and integration to a custom dataset, the instructions from [22] were followed.

Hydra was run through a Docker container with ROS2 Jazzy sourced from the official Hydra-ROS github [13]. This setup allowed smooth running in parallel with S-Graphs+ as it removes ROS version conflicts. The Docker also allowed smooth operation due to the pre-installed dependencies for things such as CUDA and TensorRT, avoiding manual dependency configuration.

The original deployment system was a personal laptop with AMD Ryzen 5000 Series with AMD Radeon Graphics, a problem arose when deploying Hydra where only co-ordinate frames were updating, with no graph being built. This was identified for a lack of NVIDIA graphics to run TensorRT and CUDA. Once identified, deployment was moved to a HCCR lab computer which had more computing power (see Table 4.7 for full specs).

Clio was deployed using a ROS Noetic Docker sourced from [17]. The decision to use a Docker was more important for this method due to the fact it needs ROS1 instead of ROS2. Whilst trying to build the workspace, an issue was identified which caused build failures. It was caused due to Clio being a ROS1 system, when some core dependencies had switched their default branches to ROS2, causing incompatibility failures. This was a known problem discussed in a community forum, which led to the discovery of a community-built container which explicitly uses the old ROS1 version of dependencies.

The setup of one native and two Dockerised methods enabled smooth transition between running each method. By only changing the active environment rather than reinstalling ROS versions, this allowed each method to run under intended conditions. This made the deployment process more practical given the time constraints of the project.

3.6.2 Robot Configuration and Data Recording

The robot configuration used to record the custom dataset was sourced from [11], which contains a differential drive robot with a Velodyne 3D LiDAR sensor, which was the baseline platform for data collection.

As only S-Graphs+ uses LiDAR data and the other methods use RGB-D, the URDF² configuration was modified to add a camera on a pole. The camera specification was set to be similar to a Realsense D435 which is widely used in robotics research, it was attached around 1.2m up the pole to simulate torso height perspective. The robot was setup to emulate the Reachy2 robot [18] style which is a realistic system that could be used for architectural tasks like building visualisation.

The robot was then teleoperated around the office scene to capture the rosbag data. This allowed control over the scene coverage and trajectory as well as simulating a realistic scenario for building visualisation.

²XML file format used in ROS to describe a robot's structure and properties

The resultant custom office rosbag was a 317s recording, which was long enough to explore multiple rooms of different sizes (to assess room segmentation). The topics recorded in the rosbag are shown in Table 3.2 below.

Table 3.2: Custom office rosbag ROS topics, their sources, and usage by methods.

ROS Topic	Source	Used By
PointCloud2	Velodyne LiDAR	S-Graphs+
Image (RGB)	Camera	Hydra, Clio
Image (Depth)	Camera	Hydra, Clio
CameraInfo (RGB)	Camera	Hydra, Clio
CameraInfo (Depth)	Camera	Hydra, Clio
Odometry	—	All methods
TFMessage	—	All methods
TFMessage (static)	—	All methods

PointCloud2 data is used by S-Graphs+ while the image data is used by Hydra and Clio. Odometry and TF is used throughout all methods for localisation.

3.6.3 Method parameters

Hydra

Hydra’s semantic segmentation was performed using the Ade20k EfficientViT-L2 model (ade20k-efficientvit_seg_l2.onnx) [23], which was selected as it is a recommended model for Hydra. The Ade20k-indoor label space was also used with this model.

The original Hydra parameters were not setup for a scene with such varying room-types like the custom office scene, where small offices or bathrooms neighbour a large office space. This led to a massive under-segmentation of the scene due to dilation distance being almost the same size as these smaller rooms. For example, the uhumans2 parameters include a 0.5m and 1.2m min and max dilation respectively, along with a 10 min room size.

A variety of parameters were tested to try and fit the behaviour to the custom office scene structure. After testing six sets of parameters, it was decided to stick with the parameters found in reachy_house.yaml. The most notable are shown in Figure 3.3.

```
room_finder:
  min_dilation_m: 0.15
  max_dilation_m: 0.3
  min_component_size: 3
  min_room_size: 5
  dilation_threshold_mode: PLATEAU
  min_lifetime_length_m: 0.05
  plateau_ratio: 0.05
  clustering_mode: NEIGHBORS
  dilation_diff_threshold_m: -1.0
```

Figure 3.3: Snapshot of main Hydra parameters which were tuned to better suit the custom office scene.

A min and max dilation of 0.15 and 0.3 respectively was chosen to ensure the smaller rooms do not get swallowed when the graph of places is dilated as discussed in 2.3.2.

Min room size was set to 5, which lowers from larger scenes like *uhumans2*, this was chosen to give the smaller rooms the best chance to be instantiated rather than being swallowed by the large open office.

The plateau ratio defines how strict the system is about committing to segmentation earlier. This was lowered in order to encourage creating rooms before they get merged into the large space.

Clio

A fundamental part for deploying Clio requires a task/region list, which represents the tasks and regions it should be searching for. The region and task lists used are shown in Figures 3.4 and 3.5 respectively. These were chosen to cater for all room types, as well as objects which are distinct and regular throughout the scene which would allow for detection analysis.

```
1 ---
2 - an open-plan office
3 - a private office
4 - a conference room
5 - a bathroom
6 - a hallway or corridor
```

Figure 3.4: Region list for the custom office scene.

```
1 clean the whiteboard:
2 count number of chairs:
3 sit at the table:
4 find a computer:
5 find a desk:
```

Figure 3.5: Task list for the custom office scene.

At the beginning, Clio was struggling to segment the different rooms and had a dictator region behaviour (where every region would be merged into a large node). This was likely due to a mismatch in between the scenes Clio was originally tuned for compared to the custom office scene.

Iterative testing helped to tune the Clio parameters to better suit the custom office scene. The final values were chosen based on observed improvements to segmentation and detection behaviour.

Voxel size was increased from 0.10m to 0.15m. This traded reconstruction quality to reduce the likelihood of objects being represented as fragmented instances.

The FastSAM confidence threshold was lowered to allow more candidate masks to be accepted per frame. In a dataset with varying coverage of different regions due to size, like a smaller bathroom, this helps to give a greater chance to produce detection that reach the CLIP embedding stage.

The score threshold for region detection was reduced in a similar way. This allowed regions that are less-visited or covered resulting in smaller embedding similarity scores to be recognised.

Object cluster size was adjusted to filter out small masks, which are likely noise and irrelevant views, and large masks, which occur when background regions merge into a single cluster.

These Clio parameters ended up prioritising recall over precision, this is useful in architectural tasks as its better for a region to be over-detected than missed.

3.6.4 Real-World Data Pipeline

To record the HCCR lab for real-world data deployment, a handheld ZED camera powered by a NVIDIA Jetson AGX Orin was used. This recording method was chosen due to its portability and ease of setup. The recording was then parsed into RGB and Depth frames with pose estimates (using code provided by Emin Safa Tok). The frames were then converted into a ros2 bag using a conversion script. Generative AI Claude (Anthropic) [1] was used to help develop this script, justification is outlined in Section xi. The script takes the data and sequentially writes them as rosbag topics. This rosbag could then be processed by the best-performing method (Hydra, as discussed in Section 4.4).

Hydra expects both a robot frame and odometry as TF transforms instead of raw odometry poses. Since the real-world data was recorded using a ZED camera, it does not have a defined robot frame. This meant that during deployment, a static transform had to be published to define this frame. As well as this, a custom node, which was developed with assistance from Generative AI Claude (Anthropic) [1] (justification is outlined in Section xi) had to be run to republish the odometry data as TF transforms.

3.6.5 Evaluation Scripts

For each of the metrics, the results were averaged over five runs and the Standard Deviation was also calculated. This allows the variance of non-deterministic systems like neural networks to be measured to determine method stability.

A custom script was developed to extract ground truth information from the custom office gazebo world file. The extracted data included room centroids for room detection, object centroids and bounding boxes for object detection and intersection over union measurements, as well as object labels in order to evaluate semantic accuracy.

Due to gazebo objects having more specific names such as 'office_chair_1', these names are normalised to the most accurate corresponding label in Hydra's label space. For example, computer to Appliance, and wastebasket to Storage. This meant that detected and ground truth objects had the same contextual labels so the confusion matrix is accurate. Label renaming was based on substring checking for regular names, for example all labels with chair in the name was renamed to just 'Chair'. This technique meant that the meaning of the labels was maintained. Label remaps can be found in the Appendix in Table [A.1](#)

For room detection, it is unrealistic to only accept exact centroids as true positives. Therefore, in the room detection logic, a true positive is classed as within 4m of a ground truth centroid. This threshold was chosen by taking into account the sizes of the rooms in the scene. Once a true positive is found, the ground truth room is removed from future checks. This stops multiple room nodes, like Clio's room shattering discussed in Section [4.3.1](#), from getting rewarded for over-segmentation. As well as this, the Z component of room centroid coordinates are ignored, this is because the custom office scene is a single-floor, therefore Z between centroids is negligible and can needlessly inflate the distance measurement.

The same logic from above is applied to object detection, however the threshold is lowered to within 1m of a ground truth centroid. This is because objects are on a smaller scale than rooms, therefore a smaller error is more meaningful in object terms. The same one object per ground truth is applied to objects to avoid multiple detections.

For [IoU](#) measurements, the closest ground truth object by centroid distance is found, then the [IoU](#) is calculated between the bounding boxes. This logic follows the assumption that the closest centroid is meant to be the same object. This saves time as calculating the [IoU](#) between each object and ground truth object would be computationally expensive.

4. Results and Evaluation

This chapter presents the quantitative simulation results and discusses why the results turned out the way they did and what it means for the methods architectural suitability. The chapter then concludes with a justification of the best-performing method and a qualitative assessment of the real-world results, followed by a discussion on the limitations of the evaluation.

4.1 Simulation Output

The following figures show the 3D Scene Graph output for each of the three methods when deployed on the custom office dataset.

4.1.1 S-Graphs+

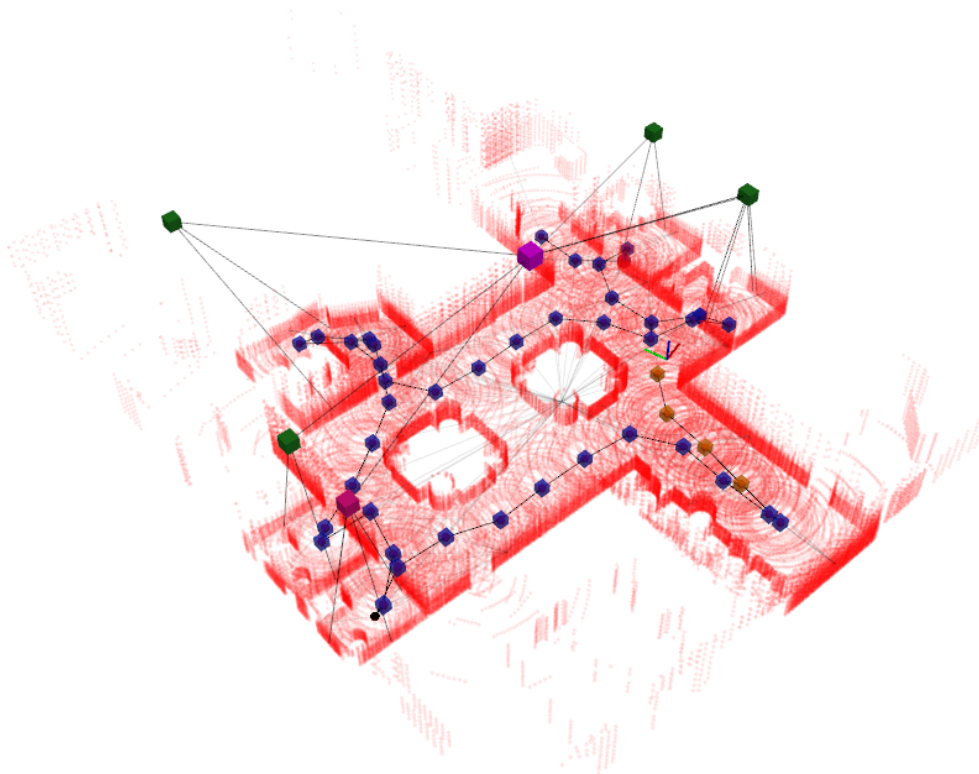


Figure 4.1: S-Graphs+ Output

4.1.2 Hydra

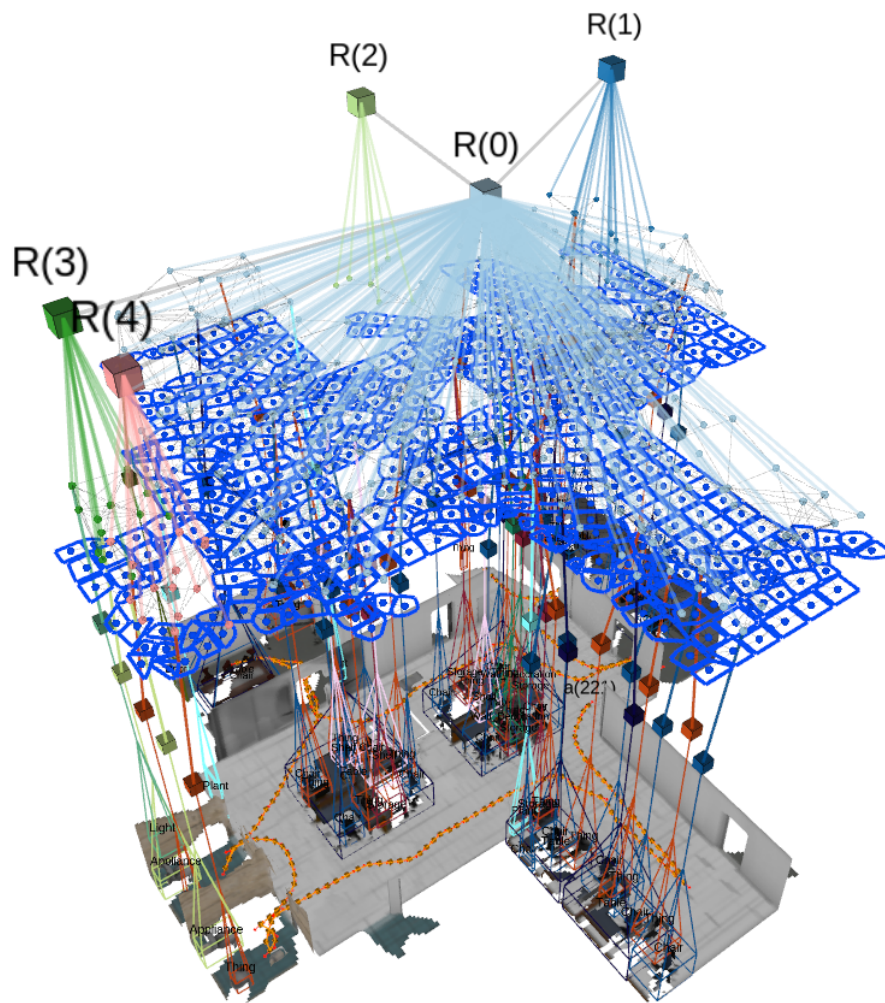


Figure 4.2: Hydra Output

4.1.3 Clio

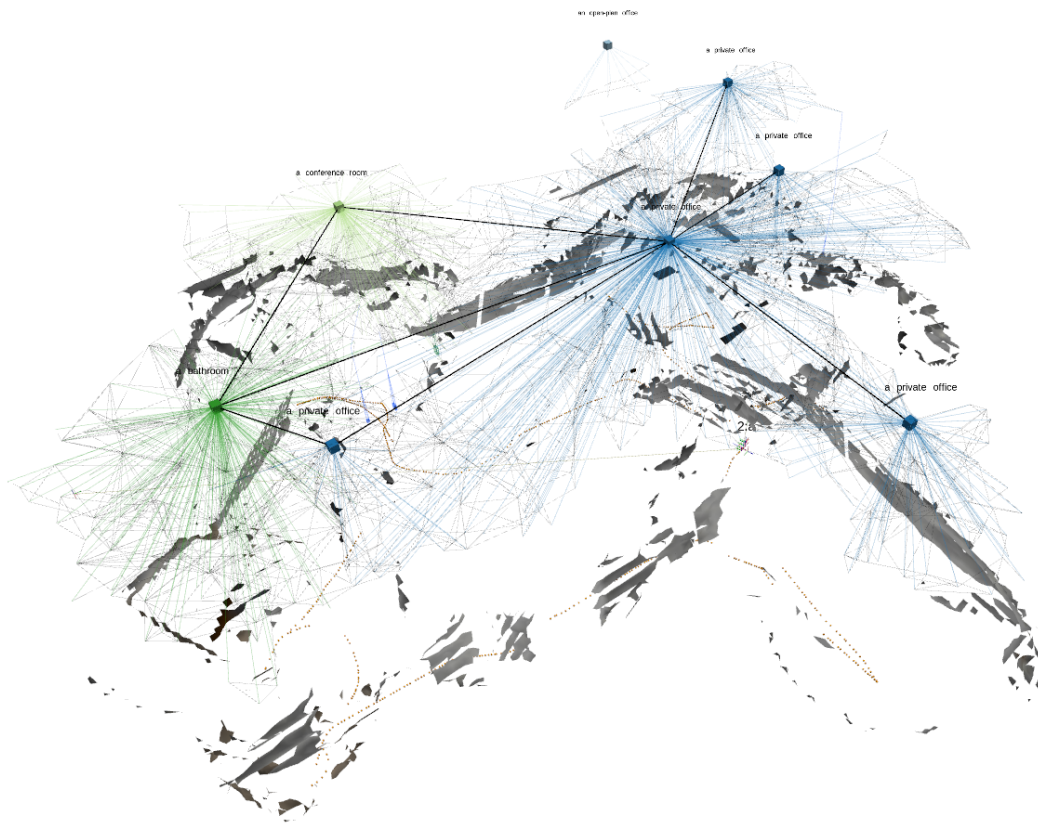


Figure 4.3: Clio Output with Mesh Layer Visible.

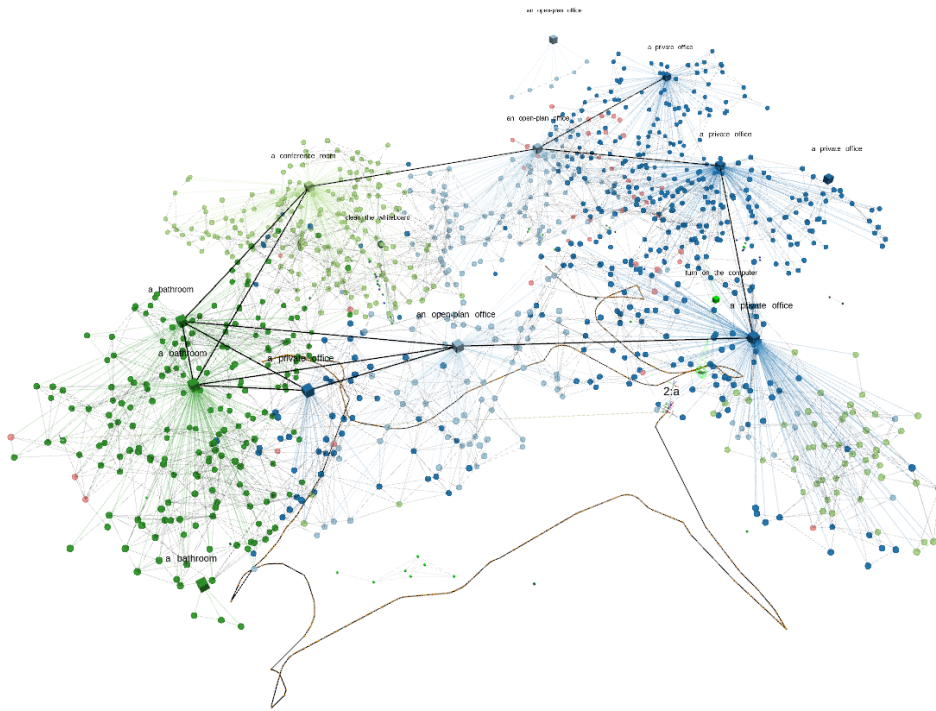


Figure 4.4: Clío Output with Mesh Layer Hidden (Different Run to Figure 4.3).

4.2 Simulation Results

All results are calculated over the 5 runs on the same dataset. **Avg represents the mean and Std represents the Standard Deviation.**

4.2.1 Structural Consistency

Table 4.1: Scene Graph Node Statistics.

Method	Total Nodes		Room Nodes		Object Nodes	
	Avg	Std	Avg	Std	Avg	Std
S-Graphs+	95.40	0.89	13.40	0.89	—	—
Hydra	1205.0	15.40	3.00	1.67	65.40	3.26
Clío	1972.0	25.67	12.80	2.32	2.20	0.40

Table 4.1 shows the different node counts for different layers in the graphs.

The data reveals a contrast between the scope of the three methods. S-Graphs+ has the most lightweight representation with only 95.40 average total nodes. This

is expected behaviour as it only focuses on free space and structural features like walls/rooms. S-Graphs+ also features a remarkably low standard deviation, which highlights the consistency of the geometric approach compared to semantic inferences.

In contrast, Hydra and Clio generate much denser graphs with 1205 and 1972 nodes. This is due to the construction of a 3D mesh as well as identifying objects within the scene. They both have higher standard deviations than S-Graphs+, which is an observed result of their neural network driven approach which can introduce fluctuations per run.

With knowledge that the scene contained 6 rooms, the Room Nodes metric provides an overview of the room segmentation performance (discussed more in Section 4.2.2). S-Graphs+ over-detected rooms with an average of 13.40. When observing the graph in Figure 4.1, one of the private office nodes shows multiple edges to the same plane, suggesting wall over-segmentation causing room over-segmentation. This is down to the geometric processing compared to the semantic/clustering approach of the other methods. Hydra on average under-segmented the scene by merging rooms into larger ones. This could be down to the specific parameters about the aggressiveness of the space clustering system. Oppositely, Clio over-segmented the scene by creating new region nodes in already tagged areas. This is most likely a result of the semantic approach to room segmentation.

The largest gap in performance is object detection. With S-Graphs+ not containing an object layer and Clio clearly struggling to identify objects with a 2.2 average count per run. Hydra stood out with 65.4 objects detected, a realistic amount for the scene.

4.2.2 Room Segmentation

Table 4.2: Room Recognition Statistics. **Best results in bold..** Per-Run Statistics in Appendix Table A.2.

Method	Recall		Precision		F1-Score	
	Avg	Std	Avg	Std	Avg	Std
S-Graphs+	0.67	0.00	0.56	0.03	0.61	0.02
Hydra	0.43	0.25	0.90	0.15	0.55	0.21
Clio	0.93	0.09	0.45	0.06	0.60	0.06

The room segmentation results outline a stark opposite in performance between Hydra and Clio. Hydra scores lowest in Recall and highest in Precision, which means when Hydra identifies a room, its likely to be a room, however it is conservative when creating new rooms. This behaviour led to the under-segmentation outlined in Section 4.2.1. Hydra also had the highest standard deviation which means it was the least

consistent for room segmentation. Clio represents the opposite behaviour with the highest recall and lowest precision, meaning it usually identifies all of the rooms in the scene, however there are lots of false positives or duplicate rooms. This leads to the over-segmentation shown in Section 4.2.1.

S-Graphs+ scored middle for both recall and precision, which allowed it to achieve the best F1 score which represents both the metrics as an overall score.

4.2.3 Object Detection

Table 4.3: Object Detection Accuracy. **Best results in bold.**

Method	Recall		Precision		F1-Score	
	Avg	Std	Avg	Std	Avg	Std
S-Graphs+	—	—	—	—	—	—
Hydra	0.83	0.01	0.49	0.02	0.62	0.02
Clio	0.00	0.00	0.00	0.00	0.00	0.00

Table 4.4: Object Spatial Accuracy. **Best results in bold.** Per-Run Statistics in Appendix Tables A.3 and A.4.

Method	Drift Over Runs (m)		Centroid mED (m)		Bounding Box IoU	
	Avg	Std	Avg	Std	Avg	Std
S-Graphs+	—	—	—	—	—	—
Hydra	0.16	0.72	0.71	0.69	0.24	0.26
Clio	2.18	2.74	3.36	0.53	0.00	0.00

S-Graphs+ is excluded from these results due to the absence of an object layer.

As previously shown in Section 4.2.1, Clio significantly struggled with detecting objects within the office scene with a recall of 0.0, despite having expected performance in the example dataset. This suggests a lack of adaptability to the custom dataset. This might be down to the differential drive motion of the camera compared to the handheld motion of the example dataset, which provides more angles of the same object (a fundamental part of Clio’s object classification).

Hydra stood out as the only acceptable method for object detection with F1 score 0.62. Also showing consistency over runs through centroid drift per run, (average 0.16m). Hydra showed acceptable accuracy through 0.71 mean Euclidean distance

between detected object and ground truth object centroids. Intersection over Union between bounding boxes revealed an average overlap of 24% between detected and ground truth.

4.2.4 Semantic Accuracy

To analyse semantic accuracy, a confusion matrix was made comparing the detected and ground truth labels of objects. The following figures are for Hydra only. Due to S-Graphs+ not supporting semantic labels, and Clio’s insufficient object detection outlined in Section 4.2.1 and Section 4.2.3, there are no analytical evidence for them.

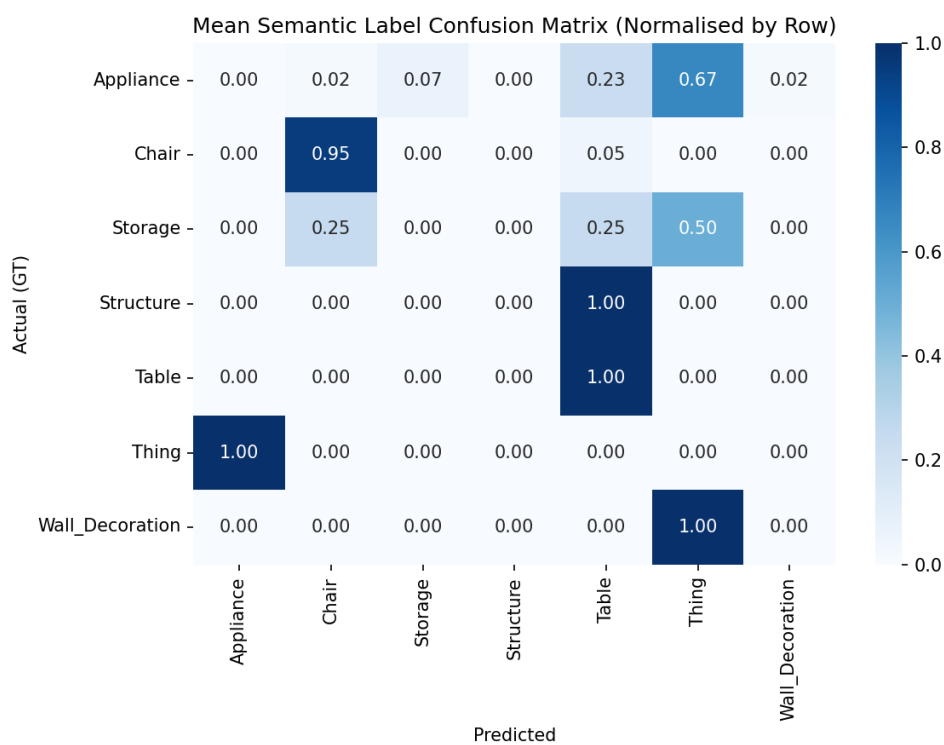


Figure 4.5: Hydra vs Ground Truth Semantic Labels: Normalised Confusion Matrix. Mean Confusion Matrix in Appendix Figure A.1.

Table 4.5: Semantic Accuracy Statistics. **Best results in bold.**

Label	Recall	Precision	F1-Score
Appliance	0.00	0.00	0.00
Chair	0.95	0.90	0.93
Storage	0.00	0.00	0.00
Structure	0.00	0.00	0.00
Table	1.00	0.18	0.30
Thing	0.00	0.00	0.00
Wall Decoration	0.00	0.00	0.00
Weighted Avg	0.39	0.34	0.35

The weighted F1-score of 0.35 from Table 4.5 shows that overall semantic labelling performance is poor, driven largely by the strong chair classification.

However, when analysing the results, it is important to consider how the label space compared to the environment affects semantic performance. The confusion matrix shown in Figure 4.5 outlines a contrasts in performance throughout Hydra’s semantic labelling pipeline. It identifies 95% of the chairs from the ground truth correctly, showing it has the potential for accurate semantics. However, it labelled 67% of ground truth “Appliances” as “Things”. This is likely due to a vocabulary mismatch between the label space and ground truth objects within the scene, as it doesn’t have labels computers or monitors which dominate the scene.

Hydra managed to correctly identify all tables (Recall 1.00). However, it massively over-labelled them (Precision 0.18), with the model often identifying Structures, storage and appliances as tables. This inaccuracy is likely due to the similarity between geometric shapes of flat sides like tables and storage containers.

There was no confusion matrix generated for Clio due to the insufficient object detection outlined in Section 4.2.1 and Section 4.2.3.

4.2.5 Practicality

Table 4.6: System Timing Comparison across Stages (Seconds). **Best results in bold.** Per-Run Statistics in Appendix Table A.5.

Method	Frontend		Backend		Total (Avg)
	Avg	Std	Avg	Std	
Hydra	0.105	0.002	0.058	0.001	0.162
Clio ¹	0.082	0.001	0.070	0.002	0.153
S-Graphs+	—	—	0.019	0.002	—

S-Graphs+ operates with a different pipeline to Hydra and Clio, therefore front-end timing data was not accessible within the project’s time-frame.

The data in table 4.6 shows a stark contrast between data approach and back-end execution time. S-Graphs+ has the smallest back-end processing time of 0.019s, significantly outperforming the other methods. This is a clear demonstration of the processing time for simply interpreting geometric data compared to image segmentation and neural networks.

The other two methods have similar performance over the total system. Clio has the fastest overall processing time with 0.153s per frame, which represents the difference between processing everything in the scene compared to only focusing on task-relevant aspects of the scene.

However, as outlined in Section 3.5.1, Clio required the rosbag to be played at 0.25x playback speed with downsampled image data to process frames without crashing. Therefore, even though the inner timing comparisons outline Clio as the most efficient, it is misleading regarding practical deployment as the system achieving real-time processing is critical.

4.3 Simulation Discussion

4.3.1 Room Segmentation:

Results from Section 4.2.2 highlighted the contrast between Hydra’s under-segmentation and the over-segmentation observed in Clio and S-Graphs+. This is likely due to the fundamentals of their room segmentation logic, where Hydra dilates a graph of places whereas Clio determines regions based on semantic clusters, while S-Graphs+ uses a geometric approach.

Geometric Plane Splitting

¹Timings are from Clio processing the rosbag at 0.25 speed

When observing the S-Graphs+ output in Figure 4.1, it shows that room nodes, notably one of the private offices, have multiple edges from the seemingly same room node to the same plane. This likely means that patchy LiDAR data lead to splitting up and over-segmenting wall planes. Since S-Graphs+ identifies rooms by associating planes, this cascaded to creating multiple room nodes consisting of small chunks of the same wall.

Dilation-Based Clustering

As discussed in Section 2.3.2, a key layer of the Hydra graph is a graph of places, which represents free space within the scene. This graph naturally contains small pathways through room connectors like doors. This graph is dilated a set amount, which removes the doorways leaving cluster of nodes (rooms).

The problem encountered during the evaluation was perfecting the dilation distance. The ideal dilation distance for segmenting large areas like office floors commonly exceeds the dimensions of smaller rooms like bathrooms, causing them to get ignored during clustering. Figures 4.6, 4.7 show this happening with the private office. At first it was recognised as a room, the graph grows a denser connection between the office and open-space, which renders the dilation distance insufficient for separating them.

The delta distance (described in Figure 2.2), is the depth of the boundary deleted before dilation to stop rooms merging. In the context of the office scene, if the dilation distance is set to segment the conference room, it is likely to cause the deletion of the smaller boundary rooms like the bathrooms, which causes the under-segmentation observed in Section 4.2.2.

Room Shattering

Clio adopts a different approach by clustering nearby object primitives based on their similarity towards a given task. However, when a room has lots of spaced out or undetected objects, it fails to cluster them and results in a shattered version of the room with multiple region nodes.

These approaches led to the results shown in Section 4.2.2, with Hydra's under-segmentation due to dilation distance, and Clio's over-segmentation due to room shattering, while S-Graphs+ over-segmented due to geometric plane splitting.

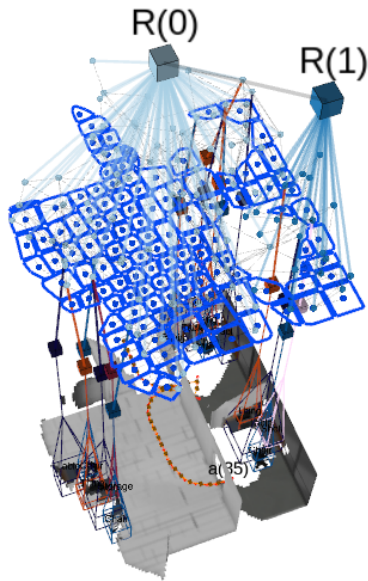


Figure 4.6: Private Office Recognised as Room at First.

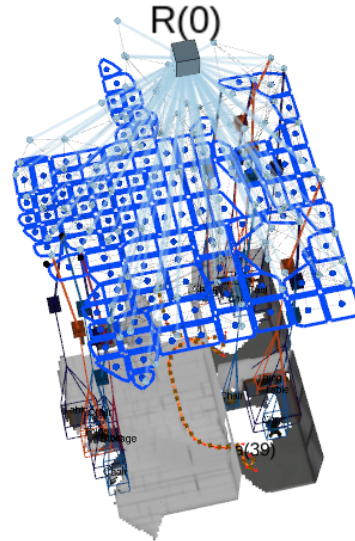


Figure 4.7: Private Office Room Node Merged into Open-Space Office.

4.3.2 Semantic Labelling

As shown in Section 4.2.4, Hydra excelled at identifying chairs, however a lot of objects within the scene were not included in the label space and hence got categorised as “Thing”. A key example of this are the computers and monitors in the office scene, which the Ade20k [23] indoor model was not trained to detect. This highlights the main limitation of Closed-Set semantics, as the model training must fit the scene in order to be accurate. Clio addresses this by using Open-Set semantics, meaning it uses provided tasks and regions in order to create labels for objects/regions. This approach therefore creates a more adaptable system in terms of semantics.

Another limitation outlined in Section 4.2.4 showed Hydra classifying flat surfaces like storage and structure as “Table”. This is likely to do with the similarities in geometrics of these types of objects as the model see straight long edges and confuses cabinets with tables, creating a geometric bias in object detection.

Clio’s object labelling could not be assessed due to insufficient object detection outlined in Sections 4.2.1 and 4.2.3. However, despite over-segmented rooms, each room node had the correct semantic label for where it was located.

S-Graphs+ was not included in semantic accuracy due to the absence of semantic labelling in its pipeline.

4.3.3 Recording Technique

As outlined in Sections 4.2.2 and 4.2.3, Clio's object detection was significantly poor compared to its expected output from the example dataset. This stark range of results suggest potential sensitivity to robot perception constraints.

The example datasets for Clio are recorded using a handheld camera maintaining short distance (approx. 1m) to things like desk cubicles etc. This recording method allows for more flexibility in terms of angles and coverage. This allows for perceiving more angles of specific objects which gives Clio more frames to initialise the object primitive. As discussed in Section 2.3.3, more frames of the same object allows a higher semantic embedding which results in a better 3D primitive. Due to the custom office scene using a differential drive robot with a fixed camera at torso height, this meant that there was a lack of angular variety compared to a camera in a human arm. Due to little variance in the object coverage, it is likely that there was insufficient CLIP embeddings to create an object node.

Some of the architectural tasks discussed before included remote visualisation and inventory. The generic robot that would be used for these tasks is likely to be similar to the setup used to record the dataset. After considering this context, instead of creating a new dataset pipeline around this limitation, it was kept to critically analyse architectural potential. If a robot had to keep wiggling around a desk to get different views of objects, this would be inefficient in terms of battery and time.

Despite the lack of in-depth coverage, Hydra still managed to output respectable results. This shows the potential for Hydra's adaptable usage.

Since S-Graphs+ uses LiDAR data, the angle and coverage of objects by a camera does not affect the accuracy of this system.

4.3.4 Practicality

As described in Section 3.5.1, Clio required the custom office scene to be played at 0.25x playback speed in order to process frames and not crash the system. Due to Clio performing flawlessly with the example dataset, this suggests a lack of adaptability to the custom office dataset.

After reviewing the example dataset, it was observed to be recorded at close range pointed at specific sections of the scene, for example office cubicles. This is a different domain to the custom office dataset which was recorded using a differential drive robot with a fixed camera joint. This limits the robot for getting close up perspectives like the ones in the example, instead supplying wide-angle views with containing more objects and segments. This forces Clio to apply semantic segmentation to multiple objects simultaneously per frame, dramatically increasing the computational load.

However, it is vital to note the difference in hardware specifications which are outlined in Table 4.7, with Clio having the biggest gap between evaluation and paper specs. This could suggest that hardware requirement had a reason for the hindering

performance.

Despite the gap in hardware, Hydra and S-Graphs+ still managed to process the dataset in real-time playback.

4.4 Best-Performing Method Justification

S-Graphs+ narrowly stood out as the most accurate for room segmentation, with F1 score (0.61) in front of Clío (0.60) and Hydra (0.55), this is expected due to the structural focus of the method. However, due to the negligible margin alongside the absence of object detection and semantic understanding, S-Graphs+ is not suitable for architectural tasks which require higher-level knowledge such as inventorying or building management.

Clío demonstrated accurate room recall with perfect semantic labelling at the room level. However, as reported in Section 4.2.3 and discussed in Section 4.3.3, Clío struggled to emulate the quality of object detection observed in the example dataset when deployed on the custom office dataset. As well as this, Clío failed at real-time processing, which was a mandatory requirement set out in Section 3.2.1. These two vital limitations exclude Clío from real-time tasks such as remote visualisation.

Hydra balanced decent precision for room segmentation (F1: 0.55), with robust object detection (F1: 0.62). Alongside this, it reflected consistency with low object drift per-run (Avg: 0.16m). These results demonstrate Hydra's capability to maintain a metric-semantic accurate graph in real-time with natural robot movement.

Comparing the strengths and weaknesses of each method, Hydra can be concluded as the most competent all-round method for architectural tasks, such as inventorying and remote visualisation. This is due to the quality of object detection and room segmentation, along with the real-time playback capabilities.

4.5 Real-World Results

This section provides a qualitative assessment of Hydra's performance on a custom recording of the [HCCR](#) laboratory.

A qualitative assessment was favoured over a quantitative one due to the difficulty in obtaining accurate ground-truth for the real-world scene, which is a common challenge in real-world evaluations. The assessment focuses on visual comparisons between the 3D scene graph and actual images of the HCCR lab.

4.5.1 Graph Images

The following figures show the Hydra 3D scene graph generated on the HCCR lab dataset.

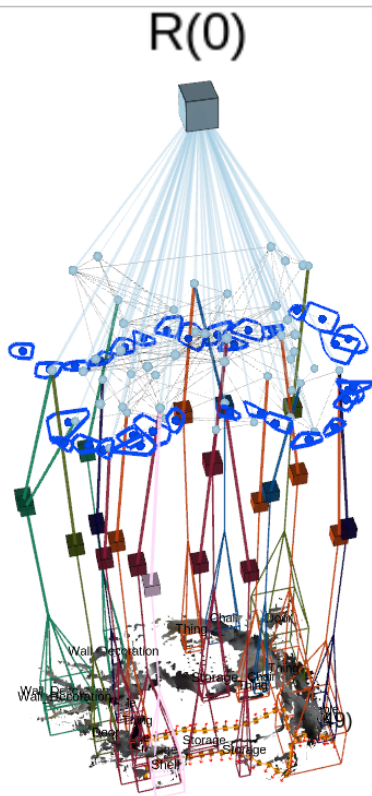


Figure 4.8: Hydra 3D Scene Graph of HCCR Lab. Including Mesh Layer

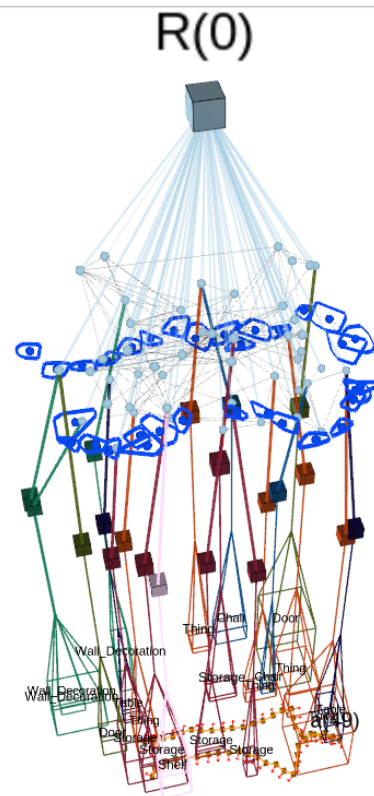


Figure 4.9: Hydra 3D Scene Graph of HCCR Lab. Without Mesh Layer



Figure 4.10: Image of Storage Cabinets in HCCR Lab.

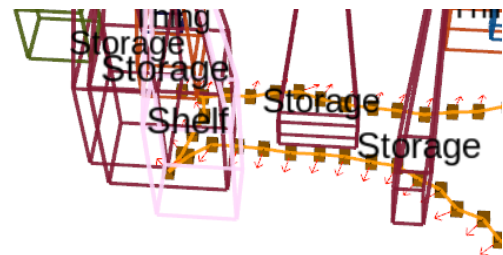


Figure 4.11: Snapshot of Hydra 3D Scene Graph of Storage Cabinets Section. (Taken from perspective of behind the wall in [4.10](#))



Figure 4.12: Image of Wall Decorations in HCCR Lab.

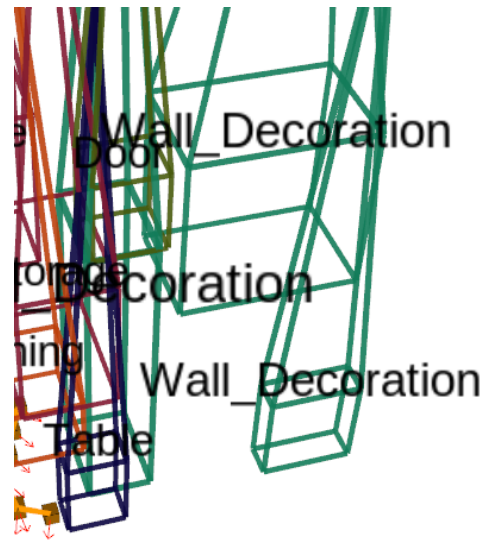


Figure 4.13: Snapshot of Hydra 3D Scene Graph of Wall Decorations Section. (Taken from perspective of behind the right wall in [4.12](#))



Figure 4.14: Image of Workstation in HCCR Lab.

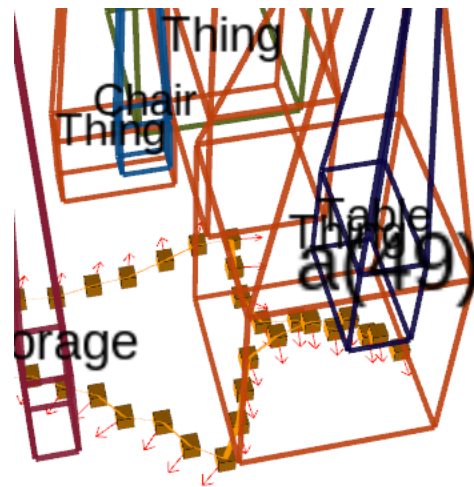


Figure 4.15: Snapshot of Hydra 3D Scene Graph of Workstation Section. (Taken from perspective of behind the right wall in [4.14](#))

4.5.2 Mesh Reconstruction

Unlike the mesh observed on simulation data shown in Figure [4.2](#), the real-world results shown in Figure [4.8](#) yielded a partially complete reconstruction, resembling a

fragmented representation of the scene. This is likely due to the gap between a hand-held camera with real-world noise, contrasted to a perfect simulation environment.

4.5.3 Object Detection / Semantics

Hydra correctly identified key objects in the scene.

Storage Area (Figure 4.10):

This corner of the lab has lots of storage cabinets along with a robot arm. As shown in Figure 4.11, it correctly identified the cabinets, and the shelf. However it didn't pickup the robot arm (behind the shelf on the table), this is likely due to the arm being occluded by the shelves, limiting coverage of it.

Wall Decorations (Figure 4.12):

Another corner of the lab contained a workstation. Figure 4.13 shows that Hydra managed to identify the posters on the wall as wall decorations, this demonstrates Hydra's ability to semantic label beyond simply geometric objects.

Workstation Area (Figure 4.14):

The workstation section of the lab contains multiple computers and different types of chairs. Figure 4.15 shows that Hydra had the same behaviour as outlined in section 4.2.4, where it classifies computers and monitors as "Thing" due to it not being in the label space. It also appears to segment all the smaller robots in the corner as one large "Thing" node, which is likely due to the limited coverage in the dataset meaning Hydra only gets a small view of all of them together.

Another notable observation is the difference in chairs, throughout the scene Hydra struggles with identifying the wheel-style chairs. However, as shown in Figure 4.15, it successfully identifies the basic chair in the centre of image 4.14. This is likely due to the data which the neural network used to label objects is trained with, as the profile of a wheeled chair differs to the standard four-legged chair.

4.5.4 Room Segmentation

Hydra correctly segmented only one room, which is correct as the laboratory is one large room containing sections like workspaces, a robot arm and storage cabinets.

4.5.5 Practicality

Similar to the capabilities exhibited in the simulation environment, Hydra was capable of processing the lab recording in real-time playback speed.

However, unlike the simulation dataset which included TF data, the ZED camera only records raw sensor data without expected ROS format. This required a static transform to be published to define the robot frame, and a custom node was written to republish odometry data as dynamic TF transforms (as Hydra processes pose via

TF rather than raw odometry). This highlights the additional setup needed when using real-world data rather than simulation.

4.5.6 Summary

Overall, Hydra demonstrated a competent integration from simulation to real-world performance. It successfully constructed a semantically labelled scene to a good recall quality, with the only difference between the two domains being the 3D mesh reconstruction. However, this is expected when moving from perfect environmental conditions to a handheld, noisy real-world environment.

4.6 Limitations

4.6.1 Dataset Context

The dataset used for the evaluation is a one-floor office scene. It provides different room types, a large room, medium room, and small rooms which are good for analysing room segmentation for varying room sizes. This provided the grounds for the discussion in Section 4.3.1.

However, the dataset only spanned one floor, which means room and floor segmentation can not be assessed, which is an entire layer in the S-Graphs+ graph (floor layer). As well as this, the evaluation only considered one usage domain due to the office context. The evaluation could have reflected different results when compared to the usage in a warehouse or hospital scene.

4.6.2 Hardware

As touched on in Section 4.3.4, the evaluation was conducted on a HCCR lab computer. Table 4.7 below shows the different specs reported in the papers for each method.

Table 4.7: Evaluation Hardware Specifications.

Method	CPU	RAM	GPU
Evaluation Specs	Intel i7-6700K (4 cores)	16 GB	NVIDIA GeForce RTX 2070
S-Graphs+	Not Reported	Not Reported	Not Reported
Hydra	AMD Ryzen 9 3960X (24 cores)	Not Reported	2x NVIDIA RTX 3080
Clio	Intel i9-13950HX (24 cores)	64 GB	NVIDIA GeForce RTX 4090 Laptop GPU

This shows the stark difference between the hardware in the paper versions and the evaluation deployments. This suggests a potential skew in the practicality results

from the evaluation, most notably the timings statistics in Section 4.2.5. The largest hardware gap is with Clio, where all the hardware is around two generations behind the expected specs, therefore this could explain the practicality problems discussed in Section 4.3.4.

However, this can be interpreted as a realistic reflection for the deployment available in the architectural industry. Some architectural companies might not have the highest-end computational hardware available, which means this evaluation reports the honest quality to be expected.

4.6.3 Recording Technique

As discussed in Section 4.3.3, the nature of the differential drive robot with a fixed camera joint limits the coverage of the specifics of the environment, like items on desks. Without testing on different recording domains, it cannot be determined whether Clio's struggles reflect system weaknesses or data input limitations.

Whilst this setup will represent a large portion of the available recording setup available in the real-world. Other configurations like Reachy2 [18], which supports an omnidirectional base with a rotatable camera would be able to provide greater coverage of the scene. This could help the Clio instantiate objects by providing multiple angles of objects. The increased coverage would also allow more LiDAR data for S-Graphs+ and more object exposure for Hydra's object detection.

4.6.4 Real-World Environment

As shown in Section 4.5.1, the HCCR lab consists of one large room. This means Hydra's capability of room segmentation on real-world data could not be analysed using this dataset. An environment with multiple rooms and floors would allow evaluation of real-world room segmentation and how the dilation problem outlined in Section 4.3.1 transfers over to real-world data.

4.6.5 Lack of S-Graphs+ Data

The evaluation metrics were designed around metric and semantic capabilities. Due to S-Graphs+ being a metric-only system, this created a disadvantage for when the whole systems were evaluated due to its lack of semantic understanding.

5. Conclusions and Future Work

This chapter summarises the main parts of project including the context, objectives, methods, results. Finishing with a discussion of the potential future work for a more in-depth evaluation.

5.1 Conclusions

The objective of this project was to identify which candidate 3D scene graph method is most suitable for assisting with architectural tasks, such as inventorying or remote building visualisation. Accurate metric data and semantic understanding is important for executing higher-level tasks as the robot needs to know more than what is geometrically around it.

The project consisted of implementing three individual environments capable of deploying the candidate methods justified in Section 3.3, which consisted of one native and two Docker environments. Each method was then deployed on a custom office dataset, recorded using a differential drive robot with a 3D LiDAR and fixed camera. The results would then allow a complete evaluation of metric and semantic accuracy as well as practicality (detailed metrics can be found in Section 3.4). These metrics were designed to reveal which method was the most deployment-ready for assisting in architectural tasks.

After analysing the results, it was determined in Section 4.4 that Hydra was the best-performing method for the architectural industry. This was due to its accuracy and efficiency under hardware constraints, which is likely to be a factor outside of academic institutions.

In comparison, S-Graphs+ was marginally superior at room segmentation, however lacks semantic understanding of the scene. Clio demonstrated strong metric-semantic room segmentation, however it was found to really struggle to instantiate object primitives, likely due to the lack of object frame captures provided by the differential drive and fixed camera used for the dataset. This was contrasted with Hydra which adapted comfortably to the perspective change. As well as this, despite expected behaviour on its example dataset, Clio had to be processed at 0.25 playback speed on the custom office dataset to prevent missing frames or crashing, meaning it was labelled incapable of real-time usage. This nullified Clio as a suitable candidate as it failed one of the mandatory requirements set out in Section 3.2.1.

As a result of this determination, Hydra was deployed on real-world data encompassing the HCCR laboratory (result shown in Figure 4.8), which proved its capability to be deployed on noisy real data and construct a 3D scene graph of an environment.

There were limitations to this evaluation. Most notably a single environment prevented comparisons over different deployment contexts such as office vs hospital. Another key limitation identified was the data collection method, which limited the coverage and views of objects, likely resulting in the Clio bottleneck observed in Section 4.3.3.

5.2 Future work

A limitation of this work was an evaluation over one scene context (an office). For a deeper evaluation over different use-cases, the methods could be evaluated over multiple scenes. For example, a warehouse would allow evaluation of detecting different types of boxes, and a hospital scene could demonstrate a humanly dense environment with dynamic parts such as bed curtains etc.

Another key limitation identified was the data collection method. The scene was recorded using a differential drive robot with a fixed camera, which provides a realistic insight into the methods likely available in the architecture world. However, a dataset recorded with a more dynamic robot such as a Reachy2 [18], which supports omnidirectional travel as well as a rotatable camera, would allow a larger coverage of the scene and objects which would give the methods more data resulting in more accurate representations.

6. Reflection on Learning

This chapter reflects on the critical challenges and turning points encountered during the project, what lessons were learnt and how they were re-applied to improve the project. Concluding with discussing the skills I will take forward into future work.

6.1 Critical narrative of the experience

6.1.1 Hardware Research

A key issue during the deployment process was a hardware mismatch when running Hydra. At first, my plan was to deploy the methods on my personal laptop containing a AMD Ryzen 7 5000 Series with AMD Radeon Graphics, this was due to portability and accessibility of a personal laptop compared to a lab desktop.

I originally deployed each method sequentially: S-Graphs+, Hydra then Clio. Due to S-Graphs+ not needing and specialist hardware, it deployed without issue on my personal laptop. However, when deploying Hydra initially, it would only output the updating frames without constructing or building a graph.

The cause was identified as a hardware mismatch, as Hydra requires a system capable of running Nvidia TensorRT in order to process frames correctly. This led me to moving to the HCCR lab computer with specs outlined in Table 4.7, which satisfied this requirement.

This experience taught me to properly research the ideal hardware specifications before starting deployment.

6.1.2 Environment Isolation

Another issue was encountered when trying to run all methods on a native environment. S-Graphs+ was originally deployed on ROS2 Humble. As outlined in the Hydra-ROS github [14], Hydra has only been tested on ROS2 Iron and ROS2 Jazzy.

Due to S-Graphs+ being reported working for ROS2 Iron also, the environment was switched to cater for Hydra compatibility. This caused some issues during integration which created a complex and messy native environment.

This became more complex later due to Clio still using ROS1 Noetic. After the decision to move to the HCCR lab computer, Dockers were used from the start when deploying Hydra and Clio which made the transition between deploying the methods easier.

This journey taught me the importance of individual deployment environments when running complex systems.

6.1.3 Community Insights

During initial Clio deployment, I used a ROS1 Noetic container which I installed from [21]. A problem was encountered during Clio installation which kept preventing the Clio workspace from building.

This problem was found to be a known issue on a community forum which identified the problem being Clio using ROS1, where many of the main branches of the dependencies had upgraded to ROS2, which caused integration problems.

A user on the community forum had created and published a Docker container which points directly to the deprecated ROS1 versions for dependencies. This community solution saved the extra work that would be needed to solve each dependency in a new container. This taught me the value of community solutions.

6.2 Analysis and application

6.2.1 Hardware Research

This issue has taught me that time should be taken into researching the hardware requirements before the deployment stage begins. I applied this lesson almost immediately as I identified that Clio requires the same TensorRT capability as Hydra. Therefore, in addition to maintaining a fair test with the same hardware, I knew to not even try to deploy it on my personal laptop first, but begin on the lab computer.

6.2.2 Environment Isolation

I began this project assuming that different versions of the same system, for example ROS2 releases, often didn't matter in terms of deployment. However this experience changed this assumption and has taught me that, especially with state-of-the-art systems, even the smallest dependency can cause cascading failures. Like with the hardware, I applied this lesson directly into my Clio deployment by beginning by building a ROS Noetic container to run Clio independently.

6.2.3 Community Insights

This experience taught me the importance of community-driven solutions. As my research led me to valuable resources which saved me considerable time in view of the time constraints of the project. This has highlighted the importance of the open-source community in efficient development processes.

6.3 Implications for future practice

In the future, I will prioritise creating individual running environments using Docker containers and virtual environments, as this gives me control and stability within systems. As well as this, I will be more mindful of hardware requirements by not prioritising ease-of-use over productivity. I will also incorporate research into community solutions into my development pipelines to ensure I am using my time efficiently by not repeating mistakes someone has already addressed.

A. Supplementary Material

This appendix contains supplementary material like tables containing raw data

A.1 Additional tables

Table A.1: Semantic Label Mapping Rules

Original Label	Mapped Class	Substring Rule	Mapped Class
table_and_chairs	Table	'chair'	Chair
office_chair	Chair	'computer'	Appliance
hangout_chair	Chair	'whiteboard'	Wall_Decoration
office_box	Storage	'toilet'	Thing
mini_fridge	Appliance	'wastebasket'	Storage
reception_desk	Table	'desk'	Table
corner	Structure	'monitor'	Appliance

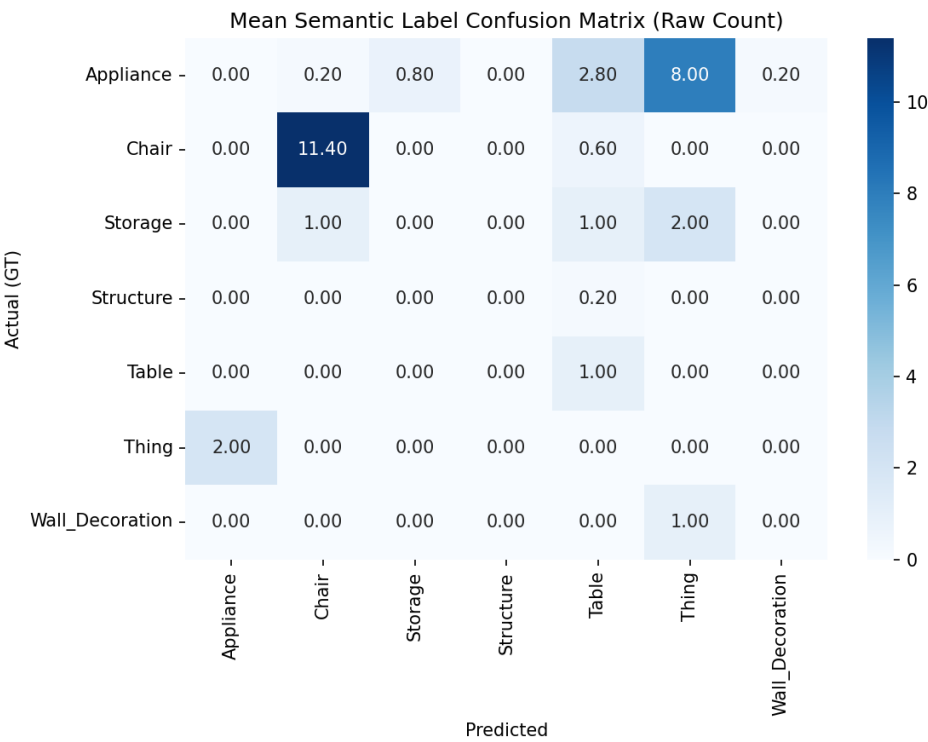


Figure A.1: Mean Count Confusion Matrix of Hydra Labels vs Ground Truth Labels

Table A.2: Raw Room Recognition Results over 5 Evaluation Runs

Run	S-Graphs+			Hydra			Clio		
	Recall	Precision	F1	Recall	Precision	F1	Recall	Precision	F1
1	0.67	0.57	0.62	0.33	1.00	0.50	1.00	0.50	0.67
2	0.67	0.50	0.57	0.83	0.83	0.83	0.83	0.50	0.62
3	0.67	0.57	0.62	0.50	1.00	0.67	0.83	0.42	0.56
4	0.67	0.57	0.62	0.17	1.00	0.29	1.00	0.35	0.52
5	0.67	0.57	0.62	0.33	0.67	0.44	1.00	0.46	0.63

Table A.3: Raw Object Spatial Accuracy (Hydra vs Clio) across 5 Evaluation Runs

Run	Hydra		Clio	
	MED (m)	IoU	MED (m)	IoU
1	0.70	0.24	3.21	0.00
2	0.73	0.23	3.51	0.00
3	0.70	0.24	3.80	0.00
4	0.70	0.25	3.66	0.00
5	0.74	0.23	2.63	0.00

Table A.4: Detailed Object Drift Statistics (Hydra vs. Clio)

Label / Category	Mean Drift (m)	Max Drift (m)	Std Dev (m)
<i>Hydra Statistics</i>			
Table	0.14	2.56	0.35
Chair	0.07	1.62	0.23
Thing	0.09	3.14	0.40
Couch	0.05	0.05	0.00
Door	0.01	0.01	0.00
Shelf	0.21	1.52	0.47
Plant	1.46	6.69	2.72
Appliance	0.01	0.02	0.01
Light	0.06	0.12	0.03
Storage	0.18	2.59	0.57
Wall_Decoration	0.03	0.09	0.02
Hydra Overall	0.16	6.69	0.72
<i>Clio Statistics</i>			
Unknown	2.18	9.22	2.74
Clio Overall	2.18	9.22	2.74

Table A.5: Raw Execution Timings across 5 Evaluation Runs (Seconds)

Run	Hydra				Clio				S-Graphs+ (Backend)	
	F_Mean	F_Max	B_Mean	B_Max	F_Mean	F_Max	B_Mean	B_Max	Mean	Max
1	0.104	0.185	0.056	0.154	0.083	0.399	0.072	0.203	0.0193	0.056
2	0.102	0.200	0.057	0.187	0.083	0.264	0.070	0.202	0.0186	0.044
3	0.104	0.207	0.058	0.162	0.083	0.313	0.072	0.231	0.0208	0.056
4	0.106	0.217	0.059	0.184	0.081	0.346	0.067	0.181	0.0167	0.047
5	0.107	0.239	0.059	0.210	0.081	0.329	0.071	0.207	0.0205	0.063

License

This work is licensed under the Creative Commons Attribution-ShareAlike License 4.0.
SPDX-License-Identifier: CC-BY-SA-4.0

You may share and adapt the work under the terms of the Creative Commons Attribution-ShareAlike 4.0 International License: appropriate credit, link to the license, indicate changes; adaptations must be shared under the same license.

Full legal code: <https://creativecommons.org/licenses/by-sa/4.0/legalcode>

References

- [1] Anthropic. Claude 3.5 sonnet, 2024. URL <https://www.anthropic.com/claude>. Accessed: April 2026.
- [2] Iro Armeni, Zhi-Yang He, JunYoung Gwak, Amir R. Zamir, Martin Fischer, Jitendra Malik, and Silvio Savarese. 3d scene graph: A structure for unified semantics, 3d space, and camera. *CoRR*, abs/1910.02527, 2019. URL <http://arxiv.org/abs/1910.02527>.
- [3] Hriday Bavle, Jose Luis Sanchez-Lopez, Muhammad Shaheer, Javier Civera, and Holger Voos. S-graphs+: Real-time localization and mapping leveraging hierarchical representations. 2022.
- [4] Melih Erdogan. Dataset-of-gazebo-worlds-models-and-maps. <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps>, 2021. Accessed: 2026.
- [5] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. Octomap: An efficient probabilistic 3d mapping framework based on octrees. *Autonomous Robots*, 34(3):189–206, 2013. doi: 10.1007/s10514-012-9321-0.
- [6] Jhoan K. Hoyos-Osorio and Luis G. Sanchez-Giraldo. The representation jensen-shannon divergence, 2024. URL <https://arxiv.org/abs/2305.16446>.
- [7] Stefan Huber. *Persistent Homology in Data Science*, pages 81–88. 01 2021. ISBN 978-3-658-32181-9. doi: 10.1007/978-3-658-32182-6_13.
- [8] N. Hughes, Y. Chang, S. Hu, R. Talak, R. Abdulhai, J. Strader, and L. Carlone. Foundations of spatial perception for robotics: Hierarchical representations and real-time systems. *The International Journal of Robotics Research*, 2024. doi: 10.1177/02783649241229725. URL <https://doi.org/10.1177/02783649241229725>.
- [9] Wassim Jabi. Topologic: enhancing the representation of space in 3d modelling environments. URL <https://www.cardiff.ac.uk/research/explore/find-a-project/view/2516263-Topologic>. Accessed: April 2026.
- [10] Wassim Jabi. topologicpy, 2025. URL <http://pypi.org/projects/topologicpy>.
- [11] Louis Le Lay. mobile-3d-lidar-sim. <https://github.com/louislelay/mobile-3d-lidar-sim>, 2024. Accessed: 2026.
- [12] Dominic Maggio, Yun Chang, Nathan Hughes, Matthew Trang, Dan Griffith, Carlyn Dougherty, Eric Cristofalo, Lukas Schmid, and Luca Carlone. Clio: Real-time task-driven open-set 3d scene graphs. *IEEE Robotics and Automation Letters*, 9(10):8921–8928, 2024. doi: 10.1109/LRA.2024.3451395.

- [13] MIT-SPARK. Hydra-ROS docker documentation. <https://github.com/MIT-SPARK/Hydra-ROS/blob/main/docker/README.md>. Accessed: March 2026.
- [14] MIT-SPARK. Hydra-ROS github repository. <https://github.com/MIT-SPARK/Hydra-ROS>, 2024. Accessed: March 2026.
- [15] Helen Oleynikova, Zachary Taylor, Marius Fehr, Juan I. Nieto, and Roland Siegwart. Voxblox: Building 3d signed distance fields for planning. *CoRR*, abs/1611.03631, 2016. URL <http://arxiv.org/abs/1611.03631>.
- [16] Open Robotics. Gazebo robot simulation. <https://gazebo.org>, 2025. Accessed: 2026.
- [17] Chad Richards. ClioDocker: Docker container enabling use of the Clio repository. <https://github.com/chadrs2/ClioDocker>. Accessed: March 2026.
- [18] Pollen Robotics. URL <https://www.pollen-robotics.com/reachy/>. Accessed: May 2026.
- [19] Antoni Rosinol, Marcus Abate, Yun Chang, and Luca Carlone. Kimera: an open-source library for real-time metric-semantic localization and mapping. *CoRR*, abs/1910.02490, 2019. URL <http://arxiv.org/abs/1910.02490>.
- [20] Lukas Schmid, Marcus Abate, Yun Chang, and Luca Carlone. Khronos: A unified approach for spatio-temporal metric-semantic slam in dynamic environments, 2024. URL <https://arxiv.org/abs/2402.13817>.
- [21] Sepideh Shakeri. Docker container with pre-installed Ubuntu 20.04 and ROS Noetic: A comprehensive guide for beginners. July 2023. URL <https://medium.com/@sepideh.92sh/how-docker-revolutionizes-application-development-a-comprehensive-guide-for-beginners-fc2d3e53eb31>. Accessed: March 2026.
- [22] SNT-ARG. LiDAR S-Graphs documentation: Installation. https://snt-arg.github.io/lidar_situational_graphs/1.installation/. Accessed: March 2026.
- [23] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ade20k dataset, 2018. URL <https://arxiv.org/abs/1608.05442>.